

Retrieval-Augmented Generation for Natural Language Processing: A Survey

Shangyu Wu

City University of Hong Kong

Ying Xiong

xiongying0929@gmail.com

Mohamed bin Zayed University of Artificial Intelligence

Yufei Cui

McGill University

Haolun Wu

McGill University

Can Chen

McGill University

Ye Yuan

McGill University

Lianming Huang

City University of Hong Kong

Xue Liu

Mohamed bin Zayed University of Artificial Intelligence

Tei-Wei Kuo

National Taiwan University Hospital

Nan Guan

City University of Hong Kong

Chun Xue

Mohamed bin Zayed University of Artificial Intelligence

Research Article

Keywords: Retrieval-augmented generation, natural language processing, vector database, large language model

Posted Date: August 22nd, 2025

DOI: <https://doi.org/10.21203/rs.3.rs-6959723/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.
[Read Full License](#)

Additional Declarations: No competing interests reported.

Retrieval-Augmented Generation for Natural Language Processing: A Survey

Shangyu Wu^{1,2}, Ying Xiong², Yufei Cui³, Haolun Wu³, Can Chen³, Ye Yuan³, Lianming Huang¹, Xue Liu², Tei-Wei Kuo⁴,
Nan Guan¹, Chun Jason Xue²

¹City University of Hong Kong.

²Mohamed bin Zayed University of Artificial Intelligence.

³McGill University, Mila.

⁴National Taiwan University.

Abstract

Large language models (LLMs) have demonstrated great success in various fields, benefiting from their huge amount of parameters that store knowledge. However, LLMs still suffer from several key issues, such as hallucination problems, knowledge update issues, and lacking domain-specific expertise. The appearance of retrieval-augmented generation (RAG), which leverages an external knowledge database to augment LLMs, makes up those drawbacks of LLMs. This paper reviews all significant techniques of RAG, especially in the retriever and the retrieval fusions. Besides, tutorial codes are provided for implementing the representative techniques in RAG. This paper further discusses the RAG update, including RAG with/without knowledge update. Then, we introduce RAG evaluation and benchmarking, as well as the application of RAG in representative NLP tasks and industrial scenarios. Finally, this paper discusses RAG's future directions and challenges for promoting this field's development.

Keywords: Retrieval-augmented generation, natural language processing, vector database, large language model

1 Introduction

Large language models (LLMs) [Touvron et al. \(2023a\)](#); [Mesnard et al. \(2024\)](#); [Jiang et al. \(2023a\)](#); [OpenAI \(2023\)](#); [Zeng et al. \(2023\)](#) have achieved significant advancements in recent years and have become the cornerstone of various applications in the field of natural language processing (NLP). These LLMs are typically pre-trained on a large amount of natural language corpus and then fine-tuned on the specific downstream tasks’ datasets. Recent works [Petroni et al. \(2019\)](#); [AlKhamissi et al. \(2022\)](#); [Meng et al. \(2022a\)](#); [He et al. \(2024\)](#) demonstrate that the success of LLMs can be explained by the fact that LLMs act as knowledge bases, which refers to implicitly storing the learned knowledge in the parameters as internal memory and generating responses by retrieving answers from memory. To store more knowledge for better generation performance, existing works generally enlarge the memory capacity by increasing the volume of parameters [Abnar et al. \(2022\)](#); [Brown et al. \(2020\)](#); [Kaplan et al. \(2020\)](#); [Hoffmann et al. \(2022\)](#).

Although existing LLMs have shown great power, several challenges still hinder the development of LLMs. One of the most prominent challenges is the hallucination problem [Ji et al. \(2023a\)](#); [Dale et al. \(2023\)](#); [Ji et al. \(2023b\)](#), which refers to the tendency of LLMs to generate responses that are coherent and fluent but factually incorrect. Another big challenge is the knowledge update issue. To update the knowledge stored in the LLMs’ internal memory [Meng et al. \(2022a\)](#); [Wang et al. \(2023c\)](#); [Zhang et al. \(2024a\)](#), it is necessary to retrain/fine-tune LLMs with new data, which is a costly process. Another challenge for general LLMs is lacking domain-specific expertise [Zhang et al. \(2023a\)](#); [Singhal et al. \(2023a,b\)](#); [Colombo et al. \(2024\)](#). Training a domain-specific LLM demands considerable manpower for dataset collection.

To address these challenges, recent works [Lewis et al. \(2020\)](#); [Borgeaud et al. \(2022\)](#); [Guu et al. \(2020\)](#) have proposed leveraging an external knowledge database to augment LLMs, known as Retrieval-Augmented Generation (RAG). By supplying LLMs with retrieved relevant factual information, the model generates outputs supported by factual information rather than relying purely on internal patterns or memorized data. As a result, the hallucination problem can be alleviated to some extent. Besides, the knowledge update issue can also be addressed by updating the external knowledge database, which can augment LLMs with up-to-date knowledge. RAG can also convert a general LLM into a domain-specific LLM by constructing and utilizing a domain-specific knowledge database. Therefore, RAG plays an important role in augmenting the functionality of LLMs, making them more accurate, knowledgeable, and reliable in a wide range of applications.

Contributions: This paper reviews all techniques involved in RAG for natural language processing. Although there are several survey papers for RAG [Li et al. \(2022a\)](#); [Gao et al. \(2023\)](#); [Hu and Lu \(2024\)](#); [Zhao et al. \(2024\)](#); [Huang and Huang \(2024\)](#); [Ding et al. \(2024\)](#), our survey still has some key insights. Table 1 shows the differences between this survey and existing surveys.

1. This paper systematically introduces each component of RAG, including details about the retriever from building to querying and techniques of the retrieval fusions with tutorial codes.

Table 1 Comparison of existing surveys [1] Li et al. (2022a), [2] Gao et al. (2023), [3] Hu and Lu (2024), [4] Zhao et al. (2024), [5] Huang and Huang (2024), and [6] Ding et al. (2024). ✓ represents that the corresponding survey has introduced the content **in detail**. Δ indicates that the corresponding survey **briefly** introduced the content. – represents that the corresponding survey **doesn’t mention** the content.

Modules		[1]	[2]	[3]	[4]	[5]	[6]	Ours
Retriever	Building	–	✓	Δ	Δ	Δ	✓	✓
	Querying	Δ	✓	Δ	Δ	Δ	✓	✓
Retrieval Fusions		Δ	Δ	✓	✓	Δ	✓	✓
Generator		Δ	Δ	✓	✓	Δ	✓	✓
Tutorial Codes		–	–	–	–	–	–	✓
RAG Training/Update		–	Δ	Δ	Δ	Δ	✓	✓
RAG Benchmarking		–	✓	✓	✓	✓	–	✓
RAG on NLP tasks		✓	Δ	✓	Δ	–	Δ	✓
RAG Tools		–	–	–	✓	–	✓	✓

2. This paper exhibits different RAG training strategies, including RAG with/without datastore update.
3. This paper further discusses RAG evaluation and benchmarking, as well as the applications of RAG on downstream NLP tasks and practical NLP scenarios.
4. This paper finally identifies promising future directions for exploring and the main challenges for addressing.

The remainder of this paper is organized as follows. Section 2 gives an overview of RAG. Section 3 and Section 4 comprehensively introduce all technical details used in retrievers and retrieval fusions. Section 6 presents how to train the RAG with/without new knowledge. Section 8 presents the techniques used in representative NLP tasks. Section 9 shows the applications of RAG in practical NLP scenarios. Section 10 discusses the future directions of RAG. Section 11 makes a final conclusion of this paper.

2 Overview of Retrieval-Augmented Generation

This section gives an overview of Retrieval-Augmented Generation (RAG) for NLP. An RAG system utilizes the external knowledge base $\mathbf{D}$ to enhance the generation system. Taking external documents as an example, $\mathbf{D}$ consists of external documents, each of which contains a set of chunks $c_i \in \mathbf{C}_i$. These chunks are transformed into vector embedding using an embedding model. When inputting a query q , which is embedded as a vector, then the retriever in the RAG system retrieves top- k chunks $R_q = \{r_1, r_2, \dots, r_k\}$ most relevant to the query q . The RAG system can use different fusion methods to fuse the retrieved chunks, which is discussed in Section 4. The overall process is formulated as follows:

$$\text{Retriever}(q, \mathbf{D}) \rightarrow R_q \quad (1)$$

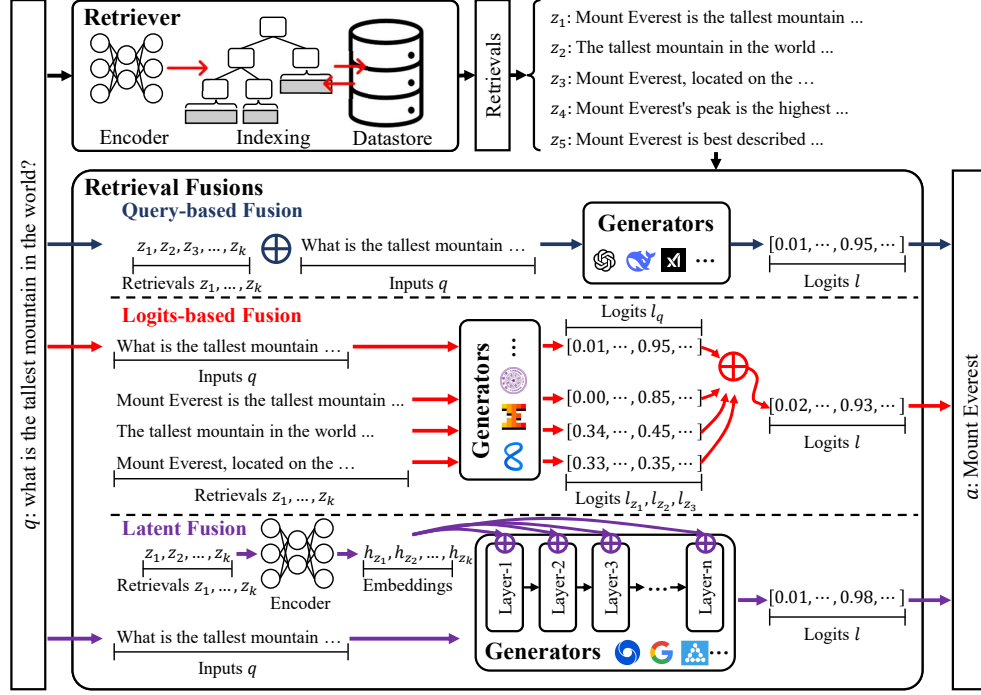


Fig. 1 The overview of retrieval-augmented generation for natural language processing. The inputs as queries are fed into both the retriever for retrieval knowledge and the generator for outputs. There are three kinds of retrieval fusions, including query-based fusion, logits-based fusion, and latent fusion.

$$\text{Generator}(\text{Retrieval Fusion}(q, R_q)) \rightarrow \text{answer} \quad (2)$$

As shown in Figure 1, RAG typically consists of three modules, the retriever, the generator, and retrieval fusions.

Retriever module usually comprises three components: an encoder for encoding inputs into embeddings, an efficient indexing that supports approximate nearest neighbor search, and a datastore for storing external knowledge in the form of key-value pairs. The main challenge in the retriever module is *finding the optimal trade-off between retrieval efficiency and retrieval quality*. The retrieval efficiency refers to how fast the relevant information can be obtained, which involves accelerating encoding, efficient indexing, batch querying in the datastore, etc. The retrieval quality refers to how relevant the information can be retrieved, which involves chunk representation learning, advanced approximate nearest neighbor search algorithms, etc.

Retrieval Fusions aims to leverage the retrieved information to augment the generation. These fusion techniques can be categorized into three major types: query-based fusion, latent fusion, and logits-based fusion. The query-based fusion augments inputs with retrievals before feeding them into the generators. The logits-based fusion focuses on the output logits of generators and fuses the retrievals logits for more robust logits. The latent fusion refers to introducing retrieval representations into the latent representations of generators, thus implicitly improving the models' performance.

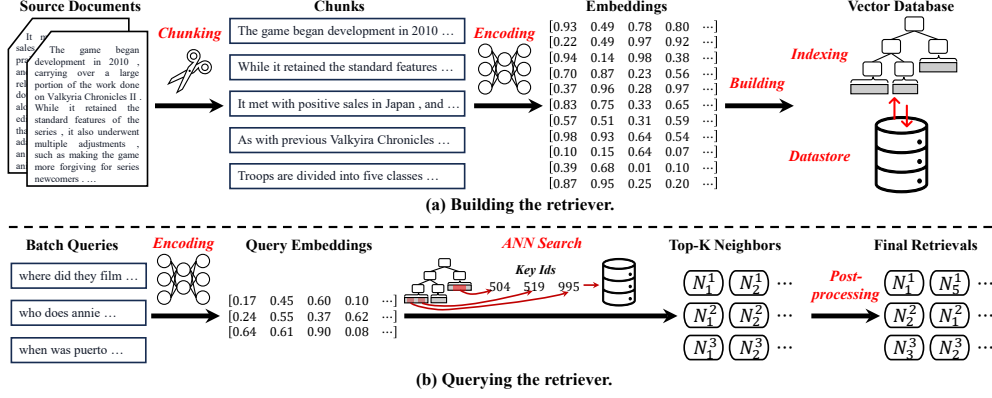


Fig. 2 Two stages of using the retriever.

Generator modules can be classified into two branches of generators: default generators and retrieval-augmented (RA) generators. The default generators include most pre-trained/fine-tuned large language models, such as GPT-series models Radford et al. (2018, 2019); Brown et al. (2020); OpenAI (2023), Mistral models Jiang et al. (2023a), and Gemini-series models Anil et al. (2023); Mesnard et al. (2024); Reid et al. (2024). The RA generators refer to the pre-trained/fine-tuned generators that consist of modules for fusing retrievals, such as RETRO Borgeaud et al. (2022); Wang et al. (2023a) and Enc-Dec Li et al. (2022b). Those generators generate responses or make predictions.

The workflow of RAG involves three steps: 1) retrieving the relevant information from external databases based on given inputs; 2) fusing the retrieved information with inputs or intermediate states based on the fusion techniques; 3) making predictions by generators based on the input and corresponding retrievals.

3 Retriever

Figure 2 shows the two stages for using the retriever, which involves first building the retriever and then querying the retriever. The following sections will introduce details about each stage.

3.1 Building the Retriever

This section will explain how to build a retriever using a large natural language corpus. As shown in Figure 2 (a), the process involves three steps: chunking corpus, encoding chunks, and building the vector database. Specifically, building the vector database includes building the ANN index and storing the data with key-value pairs.

3.1.1 Chunking Corpus

Chunking techniques generally refer to dividing large documents into small text chunks Muszynska (2016); Ishiwatari et al. (2017); Gong et al. (2020); Borgeaud et al.

(2022); [Chen et al. \(2022a\)](#), which is an indispensable key step in the process of building the retriever. The intuitions behind chunking techniques are, (1) The texts or embeddings used for the indexing should be semantically independent, containing one core idea for models to encode. Short texts are more likely to be ambiguous, for example, the word “apple“ can refer to a fruit or a company. (2) Encoding a long sequence document would result in considerable resource overheads when using existing transformer-based models, while processing shorter text chunks can significantly accelerate the encoding process and save memory costs. Therefore, the main challenge of the chunking techniques is to find the best chunking size to make a better trade-off between text semantics and encoding efficiency.

To solve the above challenge, three key points need to be considered when determining the chunking size:

1. **Task’s preference.** Different tasks may benefit from different kinds of retrieval chunks. For example, question-answer tasks may prefer short phrases, while summarization tasks may prefer long documents.
2. **Encoder’s preference.** Different encoder models have varying encoding capabilities on texts with different lengths. For example, models in the sentence-transformer [Reimers and Gurevych \(2019\)](#) behave better on a single sentence, while the text-embedding-ada-002 [OpenAI \(2022\)](#) is good at longer texts.
3. **Query’s preference.** The length of the user’s queries should be aligned with the chunking size, which implicitly aligns the amount of contextual information in chunks with that in queries, thus improving the relevance between queries and retrievals. For example, a retrieval database built on short phrases may be useless for queries with long documents.

Overall, there is no golden rule for determining the chunking size, and it depends on the specific RAG scenarios.

There are basically three types of chunking techniques, including **the chunking with fixed length**, **the semantic chunking**, and **the content-based chunking**. Chunking with fixed length is the simplest way to split documents sequentially using a length hyperparameter. The semantic chunking cuts documents based on semantics, such as the period character or the newline character that represents the end of the sentence. Existing state-of-the-art natural language processing toolkits, such as NLTK [NLTK \(2001\)](#) and spaCy [explosion \(2016\)](#), have provided convenient sentence-cutting methods. The content-based chunking segments documents according to the unique structural characteristics. For example, electronic medical records can be easily segmented based on the sections, or programming codes can be segmented based on function blocks.

3.1.2 Encoding Chunks

Encoding refers to numericalizing textual chunks as vector representations (embeddings). These embeddings generally capture the semantics of the chunks, enabling the retriever to perform similarity searches based on content relevance rather than just keyword matching.

According to the sparsity of the embeddings, there are two kinds of encoding methods, i.e., **sparse encoding** and **dense encoding**. The sparse encoding represents text

by creating high-dimensional vectors where most elements are zero. The basic sparse encoding is one-hot encoding [Harris and Harris \(2010\)](#), which represents a word with a high-dimensional vector as large as the vocabulary table size but only marks the value corresponding to the presence of the word as one. The embeddings produced by such encodings are called the one-hot vector. Other common sparse encodings include:

1. **Bag of Words (BoW)** [Harris \(1954\)](#). This encoding improves one-hot encoding by replacing the zero-one counting with the frequency counting. However, BoW ignores the syntax and word order in the documents and focuses on statistical information, thus only expressing limited semantics.
2. **Term Frequency-Inverse Document Frequency (TF-IDF)** [Rajaraman and Ullman \(2011\)](#). This encoding not only counts the occurrence (frequency) of each word but also adjusts these counts based on how common the word is across all documents (inverse document frequency). TF-IDF helps emphasize words that are more descriptive of the document’s content.
3. **BM25** [Robertson and Zaragoza \(2009\)](#) is a probabilistic ranking algorithm used in information retrieval to estimate the relevance of documents to a search query by balancing term frequency, inverse document frequency, and document length normalization, ensuring robust scoring even for long or short documents. BM25 focuses on lexical matches and is computationally efficient, making it a cornerstone of traditional search engines.

Sparse encoding is an efficient way to encode textual chunks. However, such encoding methods may not capture deeper semantic meanings well.

The dense encoding generates vectors where each dimension can capture a range of semantic features, and most elements are non-zero floating points. The dense embeddings are generally produced by (deep) neural network models,

1. **BERT** [Devlin et al. \(2019\)](#) and **Variants**. Bidirectional Encoder Representation from Transformers (BERT) is a typical pre-trained transformer model, generating dense semantic embeddings that capture the contextual information. Other BERT variants, such as RoBERTa [Liu et al. \(2019\)](#), DistilBERT [Sanh et al. \(2019\)](#), and ELECTRA [Clark et al. \(2020\)](#), further improve the semantic representations with advanced learning techniques.
2. **Siamese Encoders**. This is a type of neural network designed to learn the similarity between inputs, which is usually trained with contrastive learning. Existing state-of-the-art siamese encoders are DPR [Karpukhin et al. \(2020\)](#), SimCSE [Gao et al. \(2021\)](#), Contriever [Izacard et al. \(2022\)](#).
3. **LLM-based Encoders**. This type of encoder benefits from the powerful representation capability of LLMs. LLMs, which contain billions of parameters and are pre-trained on vast amounts of data covering a wide range of topics, have advanced semantic language understanding capabilities. Typical LLM-based encoders are text-embedding-ada-002 [OpenAI \(2022\)](#), bge-embedding [Xiao et al. \(2023\)](#), mxbai-embedding [Lee et al. \(2024\)](#), MedCPT [Jin et al. \(2023\)](#).

Compared to sparse encoding, dense encoding leverages deep neural networks, especially transformers [Vaswani et al. \(2017\)](#), to capture broader linguistic and semantic

information. Dense encodings are widely used in most representation scenarios. Moreover, some works also utilized hybrid methods to encode text for leveraging both lexical and semantic information [Lewis et al. \(2022\)](#); [Luo et al. \(2023\)](#).

3.1.3 Building the Index

Indexing in the vector database aims to accelerate the search process for data similar to high-dimensional query embedding. Unlike common indexing in databases, indexing in the vector database mainly focuses on supporting efficient approximate nearest neighbor (ANN) search [Johnson et al. \(2021\)](#); [Douze et al. \(2024\)](#); [Guo et al. \(2020\)](#); [Li et al. \(2023a\)](#); [Fu et al. \(2024\)](#) rather than transaction operations like insertion, deletion, and update. The key challenge of indexing is making a good trade-off between search quality and search efficiency. To solve the challenge, there are various specific optimizations in both algorithmic aspects and systematic aspects to be explored, including choices of similarity metrics, dimension reduction (DR) on embeddings, advanced ANN indexing, system-level optimizations, hardware-aware optimization, and so on. Due to the page limits, this section discusses the optimizations that significantly affect the search quality and efficiency.

Choice of Similarity Metrics. The similarity metrics are the basic components in the retriever, which measures the degree of relevance between query embeddings and chunk embeddings. The similarity metrics would affect the search quality. Typical similarity metrics include cosine similarity, Euclidean similarity, and Manhattan distance.

Dimension Reduction on Embeddings. Reducing the dimensionality of embeddings can improve search efficiency but at the risk of harming the semantic representations. The basic but effective dimension reduction (DR) is the principal component analysis (PCA). The PCA is a simple statistical technique that transforms the original data into a new coordinate system while retaining the most important features. Another popular and advanced dimension reduction is locality-sensitive hashing (LSH). LSH significantly reduces the dimensionality by mapping the data into buckets but preserves the similarity of the original input data. The intuition behind LSH is that the nearest neighbors will be mapped into the same buckets. Unlike LSH, product quantization (PQ) [Jégou et al. \(2011\)](#) is another popular and effective DR technique for ANN search. The core idea of the PQ is to divide the high-dimensional space into smaller, independently quantized subspaces. Each subspace creates a codebook of different quantized integers to form the representative and compact vectors. The above techniques enable efficient storage and fast approximate search but may lose semantic information. Recent work [Chevalier et al. \(2023\)](#) proposed a new technique named AutoCompressor that reduces the dimension of embeddings by compressing the original context into semantically shorter embeddings.

Advanced ANN Indexing. ANN Indexing generally refers to the methods or structures used to organize and manage data so that the approximate-nearest-neighbor search process is optimized for retrieval quality and retrieval efficiency. This paper will introduce several advanced ANN indexing techniques.

1. **The Inverted File system with Product Quantization (IVFPQ)** Douze et al. (2024) is a simple but effective indexing framework that combines two powerful techniques to enable an efficient and scalable ANN search process. The main idea of IVFPQ is first to cluster the data for coarse-grained partition and then to compress the data within each cluster into sub-vectors for fine-grained quantization. The coarse-grained clustering (the IVF component) significantly reduces the search space, while the fine-grained quantization (the PQ component) ensures a high retrieval performance.
2. **The Hierarchical Navigable Small World (HNSW)** Malkov and Yashunin (2020) uses a hierarchical graph structure to perform ANN search in high-dimensional spaces efficiently. Specifically, HNSW treats high-dimensional vectors as nodes and connects them with their nearest neighbors. The multi-layer graph structure is determined probabilistically to ensure fewer nodes at higher layers for efficient search.
3. **Tree-based Indexing** aims to organize high-dimensional vectors in tree-like structures, such as KD-Trees Ram and Sinha (2019), Ball Trees Huang and Tung (2023) and VP-Trees Liu and Wei (2015). Typical tree-based indexing is Approximate Nearest Neighbors Oh Yeah (Annoy) Spotify (2017), which uses a forest of trees built based on random projections to separate the vector space into multiple hyperplanes for efficient ANN search.

3.1.4 Building the Datastore with Key-Value Pairs

The datastore used in the vector database is a specialized database that stores and manages data as a collection of key-value pairs, where keys are the unique identifiers of high-dimensional embeddings and values are the domain-specific knowledge. Since the amount of the data stored in the datastore may be quite large, the storage engine, such as LMDB LMDB (2014) or RocksDB Facebook (2013), should be capable of efficient retrieval and data persistence. The key point in the datastore for ANN search is what should be used to be stored as values. For example, for question-answer tasks, when adding retrievals to prompts, the naive but effective way is to store the question embedding as the key and question-answer pairs as the value. This can help the generation process as retrievals are used as demonstrations for models. Recent works have proposed various state-of-the-art vector databases including the indexing and the datastore, such as Milvus Wang et al. (2021a); Guo et al. (2022), FAISS Douze et al. (2024); Johnson et al. (2021), LlamaIndex Liu (2022), etc.

3.1.5 Code Demonstrations

Algorithm 1 shows detailed steps to build the retriever. Lines 2-8 present the chunking and the encoding process for a natural language corpus containing multiple documents. In line 6, algorithm 1 takes the concatenation of the current chunk and the next chunk as the value. Notably, the choice of value can vary for different tasks. Another practical issue is that the memory cost of all keys and values may exceed the memory capacity of the server in the practical scenario. Thus, it is recommended that the keys and values persist in the storage if necessary.

Algorithm 1 Building the retriever.

Input: A natural language corpus $D = \{d_1, \dots, d_n\}$ for building the knowledge database, an encoder $\mathcal{E}$ for encoding chunks.

Output: The index $\mathcal{I}$ and the key-value store $\mathcal{S}$.

```
1:  $\mathcal{K} = \{\}, \mathcal{V} = \{\}$ ;
2: for  $d_i \in D$  do
3:    $c_i^1, \dots, c_i^m = \text{Chunk}(d_i)$ ;                                /* Split each data  $d_i$  */
4:   for  $j$  from 1 to  $m$  do
5:      $e_i^j = \mathcal{E}(c_i^j)$ ;                                          /* Encode each chunk  $c_i^j$  */
6:     Add  $e_i^j$  into  $\mathcal{K}$  and  $c_i^j + c_i^{j+1}$  into  $\mathcal{V}$ ;              /* Take next chunk as an exemplar */
7:     The  $\mathcal{K}$  and  $\mathcal{V}$  persist in the storage (e.g., SSD) if necessary;
8:   end for
9: end for
10: Build the index  $\mathcal{I}$  with  $\mathcal{K}$ ;
11: Store  $\mathcal{K}$  and  $\mathcal{V}$  into the key-value store  $\mathcal{S}$ ;
12: return  $\mathcal{I}$  and  $\mathcal{S}$ ;
```

3.2 Querying the Retriever

This section will explain how to query the pre-built retriever, which basically includes three steps as shown in Figure 2(b): encoding queries, ANN search, and post-processing.

3.2.1 Encoding Queries and ANN Search

To align with the pre-built embedding space, the retriever uses the same encoder to encode queries during the querying stage. The ANN search leverages the pre-built indexing and datastore to find similar data via ANN searching algorithms and then retrieves the corresponding values.

Searching the index refers to searching the pre-built index, finding the top-k nearest neighbors, and returning the unique identifiers of k nearest neighbors. The nearest neighbor search process depends on indexing algorithms or structures. Taking IVFPQ as an example, the search process first compares the query embedding with cluster embeddings and selects several candidate clusters for further search. Then, within each cluster, the search process performs the same product quantization on the query embedding and finds the top-k nearest neighbors based on the distance. Finally, the search process merges all nearest neighbor candidates and re-orders all candidates for the final top-k nearest neighbors.

Retrieving values from datastore fetches the corresponding values based on the nearest key identifiers.

3.2.2 Post-Processing

The post-processing involves a set of techniques after the initial retrieval step. These techniques aim to refine, enhance, or adapt the retrievals based on the specific task objectives. This section will list some typical post-processing techniques.

Algorithm 2 Query the retriever.

Input: A query input q , an encoder $\mathcal{E}$ for encoding chunks, the index $\mathcal{I}$, the key-value store $\mathcal{S}$, the parameter k .

Output: Top- k nearest neighbor knowledge.

```
1:  $e = \mathcal{E}(q)$ ;  
2:  $\{idx_1, \dots, idx_k\} = \mathcal{I}.Search(e, k)$ ; /* Search the top- $k$  nearest neighbors */  
3:  $\{v_1, \dots, v_k\} = \mathcal{S}.Fetch(\{idx_1, \dots, idx_k\})$ ; /* Fetch the values of the neighbors */  
4:  $\{v_{j_1}, \dots, v_{j_k}\} = PostProcess(\{v_1, \dots, v_k\})$   
5: return  $\{v_{j_1}, \dots, v_{j_k}\}$ ;
```

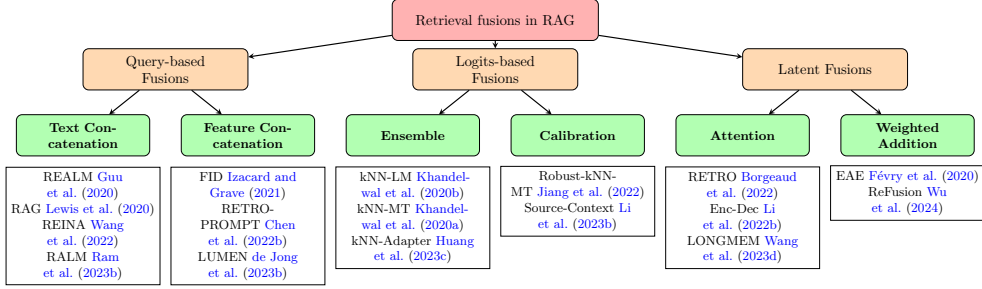


Fig. 3 The categories of fusion methods in RAG.

Reranking aims to reorder the retrieved knowledge based on task-specific objectives. The intuition is that the knowledge is retrieved based on task-agnostic metrics, such as Euclidean distance. Existing reranking methods [Chuang et al. \(2023\)](#); [Hossain et al. \(2020\)](#); [Lazaridou et al. \(2022\)](#); [Vu et al. \(2023\)](#) mostly design different architectures or strategies to reorder the retrieved knowledge.

3.2.3 Code Demonstrations

After building the retriever, this section demonstrates the detailed steps of querying the retriever to obtain the top- k nearest neighbor knowledge in algorithm 2, including encoding the query (line 1), performing the approximate nearest neighbor search (line 2), and fetching the knowledge for fusion (line 3). These three steps depend on the specific APIs of encoders, indexing, and datastore. After obtaining the top- k retrievals, optimizations for post-processing are applied (line 4).

4 Retrieval Fusions

Retrieval fusions refer to how to leverage the retrieved knowledge to improve generators' performance. Basically, there are three types of retrieval fusions: query-based fusions, logits-based fusions, and latent fusions. Figure 3 shows the detailed categorization of fusions and representative works of each retrieval fusion in RAG.

Algorithm 3 Query-based Fusions.

Input: A query input q , top- k nearest neighbor knowledge $\{v_1, \dots, v_k\}$, an encoder $\mathcal{E}_f$ and a decoder $\mathcal{D}_f$ for feature concatenation, the generator $\mathcal{G}$ for text concatenation.

Output: Generated response y .

```
1: if Use the text concatenation then
2:    $x = v_1 \oplus \dots \oplus v_k \oplus q;$           /* Concatenate neighbor texts and query */
3:    $y = \mathcal{G}(x);$ 
4: else
5:    $e_q = \mathcal{E}_f(q), e_{v_j} = \mathcal{E}_f(v_j), j \in \{1, \dots, k\};$ 
6:    $e_x = e_q \oplus e_{v_1} \oplus \dots \oplus e_{v_k};$  /* Concatenate embeddings of neighbors and query */
7:    $y = \mathcal{D}_f(e_x)$ 
8: end if
9: return  $y;$ 
```

4.1 Query-based Fusion

The simplest and most direct fusion technique is query-based fusion, which integrates the retrieved information with input queries to generate responses. The query-based fusion can be further categorized into two sub-classes according to the type of concatenated information, i.e., text concatenation and feature concatenation.

Text concatenation involves performing query-based fusion with raw texts, making it particularly suitable for contemporary LLMs like GPT-4. These models function as black-box systems with limited interaction capabilities, typically offering only API access to users. Existing works [Guu et al. \(2020\)](#); [Lewis et al. \(2020\)](#); [Ram et al. \(2023b\)](#) directly concatenate the input with the top- k retrieved sentences/documents to form the query for generators. To better use the in-context learning capability of LLMs, some works [Fabbri et al. \(2020\)](#); [Wang et al. \(2022\)](#); [Li et al. \(2023c\)](#); [Vu et al. \(2023\)](#) design effective prompt templates to integrate retrieved information and inputs. To address the issue of lengthy inputs after concatenating retrievals, recent studies [Lyu et al. \(2023\)](#); [Xu et al. \(2023a\)](#); [Arefeen et al. \(2023\)](#); [Wang et al. \(2023f\)](#); [Liu et al. \(2023a\)](#) have introduced methods for assigning importance weights to elements within the retrieved knowledge base and filtering out less relevant contexts based on these weights.

The feature concatenation involves merging the encoded retrievals with the input features. A simple yet effective approach is FID [Izacard and Grave \(2021\)](#), which first encodes the retrieved passages into sparse or dense representations and then takes the concatenated features as the input for a generator. The state-of-the-art performance of the FID demonstrates the efficacy of feature concatenation. The follow-up works [Sachan et al. \(2021\)](#); [Guo et al. \(2023\)](#); [de Jong et al. \(2023b\)](#); [Izacard et al. \(2023\)](#); [Liu et al. \(2023b\)](#) further improve the FID by jointly tuning the retriever and the encoder, which can enhance the retrieved knowledge’s representations. Besides, Chen et al. [Chen et al. \(2022b\)](#) concatenate the representations of related knowledge as demonstrations for prompt learning, yielding better generalization.

Algorithm 3 presents how to leverage query-based fusions to fuse retrieved knowledge. For those using text concatenation [Guu et al. \(2020\)](#); [Ram et al. \(2023b\)](#), algorithm 3 first concatenates the retrieved texts and inputs (line 2), then feeds the

Algorithm 4 Logits-based Fusions.

Input: A query input q , top- k nearest neighbor knowledge $\{v_1, \dots, v_k\}$, the generator $\mathcal{G}$.

Output: Generated response y .

```
1:  $y_q = \mathcal{G}(q)$ ;  
2: for  $j$  from 1 to  $k$  do  
3:    $y_{v_j} = \mathcal{G}(v_j)$   
4: end for  
5: if Use ensemble then  
6:    $y = \lambda \sum_j y_{v_j} + (1 - \lambda)y_q$ ;  
7: else  
8:    $\lambda_t = \text{Calibrate}(y_q, y_{v_1}, \dots, y_{v_k})$   
9:    $y = \lambda_t \sum_j y_{v_j} + (1 - \lambda_t)y_q$ ;  
10: end if  
11: return  $y$ ;
```

concatenated input into the generator. Notably, since there is a limit to the maximum input length of existing language models, concatenating too many retrievals would result in a truncation of the concatenated input, which may cut the given input. Therefore, designing the prompt template is the key step for this branch of work. For those using feature concatenation [Izcard and Grave \(2021\)](#); [Guo et al. \(2023\)](#), algorithm 3 first leverages an encoder to obtain the feature (line 5), then concatenates the feature of input and retrievals (line 6), finally passes the concatenated feature into a decoder model (line 7). This branch of work generally incurs high memory costs due to the long sequence length.

4.2 Logits-based Fusion

The logits-based fusion refers to incorporating the retrieved knowledge into the output layers. Basically, retrieved knowledge would be fed into the same model to obtain the logits for enhancing or calibrating the predictions. Therefore, logits-based fusion can be categorized into two branches, i.e., ensemble-based fusion and calibration-based fusion.

Ensemble-based fusion treats the logits from the retrieved knowledge as part of an ensemble of predictions. Such ensemble-based fusion can significantly improve the generalization and robustness of the model [Xiong et al. \(2023\)](#); [Khandelwal et al. \(2020b,a\)](#). One notable work of ensemble-based fusion is kNN-LM [Khandelwal et al. \(2020b\)](#), which aggregates the logits of the top- k nearest neighbors' targets and then interpolates the final predictions. Similar to kNN-LM, [Khandelwal et al. \(2020a\)](#) propose kNN-MT to enhance the machine translation using retrievals' logits, which is also followed by a branch of works [Zheng et al. \(2021\)](#); [Huang et al. \(2023c\)](#).

Different from ensemble-based fusion, calibration-based fusion uses the logits from the retrieved knowledge as a form of calibration for the model's predictions. Specifically, [Jiang et al. \(2022\)](#) propose a confidence-enhanced kNN-MT that

Algorithm 5 Latent Fusions.

Input: A query input q , top- k nearest neighbors $\{v_1, \dots, v_k\}$, the encoder $\mathcal{E}$, the generator $\mathcal{G}$ containing l pairs of modules $\{(\mathcal{M}_1^A, \mathcal{M}_1^F), \dots\}$, where $\mathcal{M}_i^A$ and $\mathcal{M}_i^F$ are the attention module and the FFN module at layer i , $\mathcal{M}_i^C$ is the cross-attention module used in attention-based latent fusions.

Output: Generated response y .

```
1: if Use the attention then
2:    $h_0^F = q$ ;
3:   for  $i$  from 1 to  $l$  do
4:      $h_i^A = \mathcal{M}_i^A(h_{i-1}^F)$ ;
5:      $e_{v_1}, \dots, e_{v_k} = \mathcal{E}(v_1, \dots, v_k, h_i^A)$ 
6:      $h_i^R = \mathcal{M}_i^C(h_i^A, e_{v_1}, \dots, e_{v_k})$ ;          /* Cross-attention */
7:      $h_i^F = \mathcal{M}_i^F(h_i^R)$ 
8:   end for
9:    $y = LM\_HEAD(h_l^F)$ 
10: else
11:    $e_{v_1}, \dots, e_{v_k} = \mathcal{E}(v_1, \dots, v_k)$ 
12:    $h_0^F = q$ ;
13:   for  $i$  from 1 to  $l$  do
14:      $h_i^A = \mathcal{M}_i^A(h_{i-1}^F)$ ;
15:      $h_i^R = h_i^A + \frac{1}{k} \sum_j w_j e_{v_j}$           /* Weighted sum */
16:      $h_i^F = \mathcal{M}_i^F(h_i^R)$ 
17:   end for
18:    $y = LM\_HEAD(h_l^F)$ 
19: end if
20: return  $y$ ;
```

refines the kNN distribution and interpolation weights with the neural machine translation confidence. Li et al. [Li et al. \(2023b\)](#) propose to leverage the source context to calibrate the retrieval-augmented neural machine translation.

Algorithm 4 demonstrates the detailed steps of using the logits-based fusion to integrate the retrieved knowledge. This branch of work first treats retrievals as similar data to augment the model (lines 2-4). For ensemble, algorithm 4 leverages a hyperparameter to fuse the retrieval logits and the output logits (line 6). For calibration, algorithm 4 dynamically determines the parameter based on the retrieval logits and the output logits (line 8). Then, algorithm 4 performs the same fusion with the computed parameter (line 9).

4.3 Latent Fusion

The latent fusion investigates merging the retrieved knowledge into the hidden states of generators for a better generation. Based on the introduction method, latent fusion can be further classified into two categories: attention-based and weighted-addition.

One notable contribution of attention-based fusion is the Retrieval-Enhanced Transformer (RETRO) [Borgeaud et al. \(2022\)](#). RETRO represents a pioneering effort in pre-training retrieval-based LLMs, introducing a new cross-attention module to integrate retrieved knowledge directly into the model's hidden states. A significant

finding from this work is demonstrating a scaling law for the retrieval database, where RETRO, with a 2 trillion token database, attains performance comparable to that of major models like GPT-3 and Jurassic-1, albeit with 25 times fewer parameters. Customizing the transformer model in RETRO highlights the potential of pre-trained, retrieval-enhanced architectures in improving the efficiency and scalability of LLMs.

In addition to RETRO, other studies [Cai et al. \(2021\)](#); [Wu et al. \(2022\)](#); [de Jong et al. \(2022\)](#); [Li et al. \(2022b\)](#); [Wang et al. \(2023g\)](#) have contributed to the field by leveraging new attention modules to introduce external knowledge. Typically, [Li et al. \(2022b\)](#) have extended the RETRO model by decoupling the context encoding from the model inference. [Wu et al. \(2022\)](#), [Wang et al. \(2023d\)](#) store the hidden attention keys and values into external memory and retrieve the knowledge from the memory using an attention mechanism.

Due to the high complexity of the attention mechanism, another branch of work adopts lightweight (weighted) additions to introduce retrieved knowledge. [Férvy et al. \(2020\)](#) propose the EAE model that retrieves top- k related entities’ embeddings from a learnable external memory and adds entities’ embeddings to the hidden states of the model. [Wu et al. \(2024\)](#) propose ReFusion, which explores various learnable reranking schemes to first re-weight the retrieved knowledge’s embeddings, then use weighted addition to incorporate them into the hidden states of the model. Those approaches signify a growing trend towards models that dynamically select and integrate relevant knowledge, paving the way for more sophisticated and nuanced language generation and understanding.

Algorithm 5 shows the steps of using latent fusion to introduce the retrieved knowledge into the hidden states of the generator. For attention-based latent fusion, algorithm 5 first encodes the retrievals with the output states of the attention module (line 5), then uses a cross-attention module to fuse the retrieval features into the hidden state (line 6). Different from attention-based latent fusion, weighted-addition-based latent fusion adopts a more lightweight way to incorporate retrieved knowledge (lines 10-19). Algorithm 5 first encodes the retrievals before feeding them into the generator (line 11), which can be done offline and directly stored as values in the data-store. Then, algorithm 5 learns a set of weights to add the retrieval features on the hidden states of generators (line 15).

4.4 Comparison of Different Fusion Methods

Each fusion method exhibits distinct advantages and limitations. The query-based fusion method is a simple and straightforward approach that preserves the raw information from retrieved information. However, it significantly increases the input sequence length, resulting in higher computational costs. Additionally, this kind of method suffers from limited contextual integration capabilities and scalability, as it struggles to effectively combine retrieved information with the input context.

The logits-based fusion method combines the output logits (or probabilities) of the base LLM with those from a separate model (or shared LLM) that processes the retrieved information. This approach is computationally efficient, as it can process retrievals and inputs in batches, avoiding increasing the input sequence length. It decouples the retrieval and generation processes, enabling independent improvements

to each component. Nevertheless, this kind of method is limited by its shallow contextual integration, as fusion occurs only at the output level. Consequently, it may underperform in complex reasoning tasks that require fine-grained interaction between the input and retrieved information.

In contrast, the latent fusion method integrates retrieved information directly into the hidden states (or intermediate representations) of the LLM. This approach is both scalable and efficient, as it avoids expanding the input sequence length. Moreover, it facilitates deep, context-aware integration of retrieved information and supports fine-grained control over how the information is fused. However, the latent fusion method necessitates significant architectural modifications and careful training to achieve optimal performance. In summary, the choice of fusion method depends on the specific application, available resources, and the desired trade-off between simplicity and performance. Each method offers unique strengths and weaknesses, making them suitable for different use cases and operational constraints.

5 Generators

This section introduces representative generators, which are generally pre-trained on large datasets. Existing generators are mostly large language models that adopt or modify the transformer-based architecture [Vaswani et al. \(2017\)](#). For example, DeepSeek [Guo et al. \(2025\)](#), Llama-series models [Touvron et al. \(2023a,b\)](#), GPT-series models [Radford et al. \(2018, 2019\)](#); [Brown et al. \(2020\)](#); [OpenAI \(2023\)](#), and Gemini-series models [Anil et al. \(2023\)](#); [Reid et al. \(2024\)](#); [Mesnard et al. \(2024\)](#) remove all encoder modules, retaining only the decoder module, which includes an attention module and a feed-forward network module. Other advanced techniques, such as root mean square layer normalization [Zhang and Sennrich \(2019\)](#), rotary position embedding [Su et al. \(2024\)](#), and group query attention mechanisms [Ainslie et al. \(2023\)](#), have been incorporated into the design of existing large language models to enhance their performance.

Existing generators can be generally categorized into two groups, close-sourced LLMs [Brown et al. \(2020\)](#); [OpenAI \(2023\)](#) and open-sourced LLMs [Guo et al. \(2025\)](#); [Touvron et al. \(2023a,b\)](#). The formers can only adopt query-based fusions to introduce external knowledge. While the latter can adapt to all kinds of fusions. Especially generators using the latent fusion typically would train the generators from scratch. Those generators integrate novel modules designed for retrieved information fusion, such as cross-attention-based module [Borgeaud et al. \(2022\)](#); [Wu et al. \(2022\)](#); [Li et al. \(2022b\)](#). Their training paradigm combines pre-training on massive textual datasets with integrating external retrieved knowledge to establish robust information grounding. As detailed in Section 4.3, those generators enable dynamic, context-aware retrieved information incorporation while further improving the LLM’s generative capabilities.

6 RAG Training and Datastore Update

This section introduces RAG training, which can be categorized into two main classes: **RAG without datastore update** and **RAG with datastore update**. The former

refers to the case where only trainable parameters in each module of RAG would be updated, and the knowledge in the datastore would remain the same during the training stage. The latter refers to the case where the knowledge in the datastore would be updated, then each module’s parameters in RAG would be updated in a similar way as the former case.

6.1 RAG without Datastore Update

The goal of training RAG without datastore update is to update the knowledge stored in the short-term memory of generators based on the existing knowledge datastore. As shown in Figure 4 (a)-(c), there are three training cases, i.e., training the retriever, training the generator, and jointly training the retriever and generator.

6.1.1 Training retriever.

Considering the case of no datastore update, training the retriever generally refers to training the retriever encoder and rebuilding the indexing. Since sparse encodings rely on statistical methods without parameters, training the encoder pertains only to dense encoding methods. Different training methods may have different goals, such as improving the semantic representations, accelerating the encoding process, or learning the domain-specific representations. The first two goals are often achieved by replacing the original encoder with a more powerful or tiny encoder, such as DistilBERT [Sanh et al. \(2019\)](#), or TinyBERT [Jiao et al. \(2020\)](#). The last requires training the original encoder on the domain-specific corpus. REPLUG [Shi et al. \(2023\)](#) updated the retriever by minimizing KL divergence between retrieval and LM scores, and asynchronously refreshing the datastore index. After training the retriever encoder, the vector database’s embeddings that serve as keys will also change. Thus, all indexes should be rebuilt with new embeddings. Besides, if the encoder remains unchanged, the indexing can be updated using new ANN searching algorithms or re-tuning the hyperparameters. After the retriever is trained, it can be directly incorporated into the RAG without updating the generator.

6.1.2 Training generator.

Training the generator involves updating its parameters or those in the retrieval fusion modules. Since the generator is generally an LLM, training the LLM is a resource- and time-consuming process. Fortunately, several parameter-efficient fine-tuning techniques, such as LoRA [Hu et al. \(2022\)](#), are proposed to address the fine-tuning problem of LLMs. Although the parameters in the retrieval fusion modules are less than those in the generator, only fine-tuning those parameters may encounter some training problems, such as low convergence and overfitting. Jointly tuning the parameters in the generator and the retrieval fusion modules is a better way to train the generator and the retrieval fusion modules if there are sufficient and powerful resources.

6.1.3 Jointly training the retriever and generator.

Apart from independently training the retriever and the generator, jointly training the retriever and the generator can be another good choice for better performance

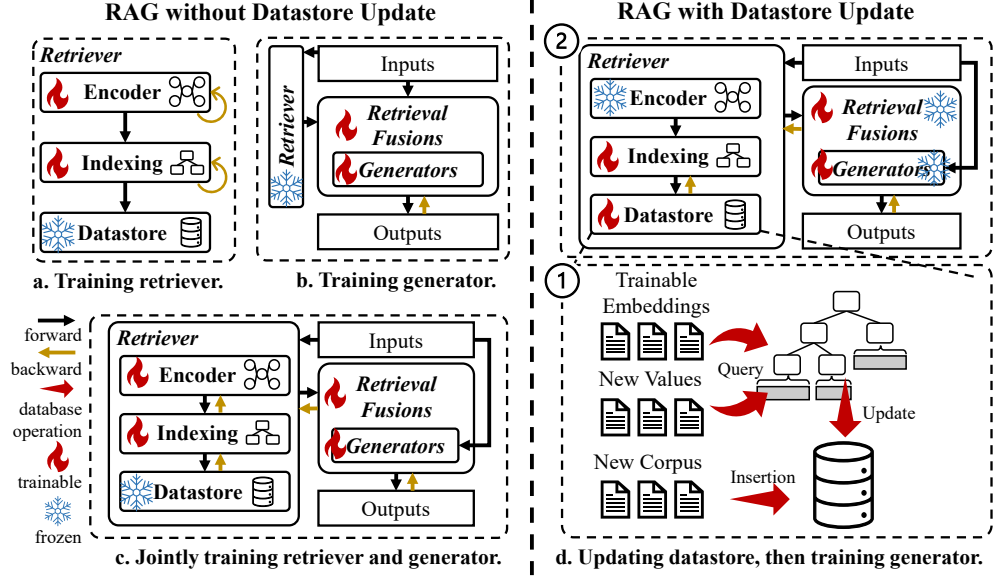


Fig. 4 Different RAG training strategies with/without datastore update.

on downstream tasks. The primary challenges involve computational complexity from large-scale retrieval and marginalization over latent documents, alongside training instability due to potential retrieval collapse and interdependent retriever-generator feedback loops. Additionally, sparse gradients from discrete retrieval steps and approximation errors from fixed top- k document selection limit effective end-to-end optimization. Typically, complex indexes, such as FAISS [Douze et al. \(2024\)](#), are not a suitable choice during the fine-tuning stage.

Existing works generally leverage the complex indexes to pre-select a small subset of nearest neighbors as candidates, then choose the final top- k nearest neighbors by performing the matrix-multiplication operations. REALM [Guu et al. \(2020\)](#) and RAG [Lewis et al. \(2020\)](#) both unify retriever and language modeling by treating retrieved documents as latent variables and optimizing end-to-end via gradient descent. They both used Maximum Inner Product Search (MIPS). RAG [Lewis et al. \(2020\)](#) only updated the encoder in the retriever, but REALM [Guu et al. \(2020\)](#) also asynchronously refreshed the indexing. Atlas [Izacard et al. \(2023\)](#) jointly pre-trained the retriever and generator, using different loss functions (such as attention distillation and perplexity distillation) to optimize the retriever. Besides, it also asynchronously refreshed the indexing. Experimental results in these works demonstrate that joint training is an end-to-end optimization that can lead to better coordination between the retriever and the generator and improve the contextual understanding of the generator.

6.2 RAG with Datastore Update

As shown in Figure 4 (d), the scenario involves two stages: updating the knowledge database, then training the retriever and the generator. There are three cases for updating the knowledge database, i.e., updating with trainable embeddings, updating with new values, and updating with new corpus. In the first case, values generally are trainable embeddings and are simultaneously/asynchronously updated with parameters in the RAG [Chen et al. \(2022b\)](#). The last two cases usually refer to updating the knowledge database with up-to-date information. Taking question-answer corpus as an example, updating with new values refers to updating the answer to existing questions, while updating with new corpus refers to adding new question-answer pairs. To update the value of existing keys requires first querying the existing key-value pairs and then performing in-place updates. For a new corpus, the datastore first needs to perform insertion operations, then rebuilds or updates the indexes as new keys are added. After updating the datastore, training the retriever and the generator is similar to RAG without datastore updates. However, this training step is not always necessary, thanks to the in-context learning capability of LLMs.

7 RAG Evaluation and Benchmark

Retrieval-Augmented Generation (RAG) systems integrate the capabilities of large language models (LLMs) with external knowledge retrieval to advance text generation tasks in natural language processing (NLP). Evaluating RAG systems requires a dual focus on retrieval quality and generation performance to ensure the system effectively leverages external knowledge and produces accurate, contextually appropriate outputs. Recent research has introduced domain-specific benchmarks and comprehensive evaluation frameworks to address these challenges.

For retrieval quality, Pipitone et al. [Pipitone and Alami \(2024\)](#) proposed a benchmark tailored for RAG systems in the legal domain, employing key retrieval metrics such as Precision@K and Recall@K to assess the relevance and ranking of retrieved documents. Adlakha et al. [Adlakha et al. \(2024\)](#) expanded the evaluation scope by incorporating correctness, faithfulness, and human evaluation to assess retrieved information, alongside novel token-overlap metrics for finer-grained analysis.

On the generation side, Hui et al. [Hui et al. \(2024\)](#) introduced a benchmark suite for RAG systems in real-world document analysis, utilizing metrics such as accuracy, F1 score, and exact match score to evaluate answer quality. Similarly, Xiong et al. [Xiong et al. \(2024\)](#) developed a medical RAG benchmark to systematically compare the performance of medical RAG systems, emphasizing domain-specific evaluation criteria.

Recent advancements have further enriched the evaluation landscape by proposing more systematic and comprehensive methodologies. Chen et al. [Chen et al. \(2024\)](#) identified four fundamental RAG abilities—noise robustness, negative rejection, information integration, and counterfactual robustness—as key evaluation dimensions. Saad-Falcon et al. [Saad-Falcon et al. \(2024\)](#) introduced ARES, an automated RAG evaluation system, which assesses RAG systems using context relevance, answer faithfulness, and answer relevance. Building on this, Friel et al. [Friel et al. \(2024\)](#) extended

the evaluation framework by incorporating additional metrics such as context utilization and answer completeness, providing a more holistic assessment of RAG system performance. These efforts collectively highlight the growing emphasis on rigorous, domain-specific, and multi-dimensional evaluation methods to ensure RAG systems meet the demands of diverse applications while maintaining high retrieval and generation quality standards.

8 Tasks

This section lists several classical tasks in the NLP domain and introduces advanced RAG techniques used to solve these tasks.

8.1 Language Modeling

Language modeling is the task that requires the prediction of the probability distribution of the next word or character given a sequence of words or characters, which is also named the next-token prediction task. Language modeling has become the fundamental task for pre-training large language models, which can measure the models' generation capability using the perplexity metric. The formal definition is as follows: given such a sequence of tokens $x_1, \dots, x_n$ called *Prefix*, the language modeling task aims to model its probability via next-token prediction,

$$p(x_1, \dots, x_n) = p(x_1) \cdot \prod_{i=2}^n p(x_i | x_1, \dots, x_{i-1}), \quad (3)$$

where the conditional probabilities $p(x_i | x_1, \dots, x_{i-1})$ are modeled by a parameterized language model.

Recent works mainly leverage RAG further to improve language modeling capability in the pre-training stage. A branch of works [Borgeaud et al. \(2022\)](#); [Li et al. \(2022b\)](#); [Wu et al. \(2022\)](#); [Wang et al. \(2023d\)](#) modifies the architecture of generators by adding a new cross-attention module in each transformer block for introducing retrieval knowledge. The intuition of those works is that given the similar *Prefixes* and their next tokens (retrieving stage), the pre-trained model can calibrate the model's prediction using the cross-attention module to capture the pattern between the next token and prefix (model forwarding stage). [Zhong et al. \(2022\)](#) propose to augment the language model with three types of retrieval memories/databases (local memory, long-term memory, and external memory) and optimize the next-token probability distribution with nearest neighbors retrieved from the memories/databases. Another branch of works [Khandelwal et al. \(2020b\)](#); [Huang et al. \(2023c\)](#); [Xu et al. \(2023b\)](#); [Ram et al. \(2023b\)](#); [Guu et al. \(2020\)](#) focuses on augmenting the inputs or outputs of generators with retrievals. [Guu et al. \(2020\)](#) and [Ram et al. \(2023b\)](#) concatenate the retrieved knowledge with inputs and feed the retrieval-augmented inputs into the generators. Other works [Khandelwal et al. \(2020b\)](#); [Huang et al. \(2023c\)](#); [Xu et al. \(2023b\)](#) fuse the logits of inputs as well as retrievals at the final output layer and generate the final probability distribution based on the interpolated

results. Those works believe that the concatenated/fused retrievals can provide useful context information on inputs/outputs to improve models’ robustness during the pre-training stage. Besides, Doostmohammadi et al. [Doostmohammadi et al. \(2023\)](#) focus on pre-training models with a semantic retriever (BM25) and achieve a better language modeling performance.

8.2 Machine Translation

Machine translation (MT) leverages computational linguistics algorithms to translate text or speech from one language to another automatically. The goal of MT is to produce an accurate and fluent translation, preserving the meaning of the original text while adhering to the grammatical and stylistic norms of the target language. MT systems have evolved from rule-based machine translation (RBMT) to statistical machine translation (SMT) and, more recently, to neural machine translation (NMT). In particular, NMT methods have significantly improved translation quality by leveraging deep learning techniques, which thus will be the focus of this section.

RAG techniques can further enhance MT by incorporating external knowledge into the translation process. The simplest way is to concatenate the similar translation examples into the inputs or fuse the logits of similar translation examples at the output layer. For example, some works [Wang et al. \(2022\)](#); [Cheng et al. \(2023b\)](#) retrieve similar translations according to the source text and concatenate corresponding target texts or pairs of source and target texts as examples into inputs. Other works [Hossain et al. \(2020\)](#); [Khandelwal et al. \(2020a\)](#); [Zheng et al. \(2021\)](#) feed the retrieved source text into the models and obtain the logits of the next target tokens, then aggregate all logits to generate the final predictions. Moreover, Jiang et al. [Jiang et al. \(2022\)](#) and Li et al. [Li et al. \(2023b\)](#) use the logits of retrieved examples to calibrate the aggregated logits, improving the robustness of the generation. Another branch of works [Zhu et al. \(2023\)](#); [Zhong et al. \(2022\)](#) injects external knowledge into the objective function during the training stage, refining the representation space with similar translations. Besides, Cai et al. [Cai et al. \(2021\)](#) encode similar translations and store them as the translation memory, then introduce the knowledge from memory with a cross-attention module. Instead of improving the performance, a branch of work focuses on accelerating the generation efficiency on MT tasks, such as searching from a pre-built subset [Meng et al. \(2022b\)](#); [Deguchi et al. \(2023\)](#) or a dynamic datastore [Dai et al. \(2023\)](#), searching by chunks [Martins et al. \(2022\)](#).

8.3 Text Summarization

Text summarization is the process of condensing a larger text document into a shorter version, preserving key information and the overall message. This task can be broadly categorized into two types: extractive summarization, which involves selecting and compiling parts of the original text, and abstractive summarization, which entails rewriting the essence of the text in a new, concise form. The goal is to produce a coherent and fluent summary that encapsulates the most critical information from the source material.

RAG techniques can significantly enhance text summarization tasks by leveraging external knowledge and similar documents to inform the summarization process. Wang et al. (2022); Fan and Gardent (2022); Li et al. (2023c); Cheng et al. (2023b) simply concatenates the retrieved similar summaries into inputs to generate summarizations. Instead of concatenating texts, other works fuse features at the intermediate layers by cross-attention Bertsch et al. (2023), or at the output layers by logits ensemble Hosain et al. (2020). Besides, Jiang et al. Jiang et al. (2023b) argue that retrieving for every generation may not always be the best choice and propose to retrieve external knowledge during the generation process adaptively.

8.4 Question Answering

Question Answering (QA) is a fundamental task in NLP that involves building systems capable of automatically answering human questions in natural language. QA systems can be broadly classified into two categories: open-domain, where the system answers questions about virtually anything, and closed-domain, focusing on a specific area of knowledge. The primary challenge in QA is understanding the question’s intent and retrieving accurate, relevant information from a vast collection of data to provide a concise answer. Due to the page limits, this paper only discusses the works of open-domain QA systems.

RAG techniques combine information retrieval with model-based generation, which is highly suitable for QA systems. In particular, open-domain QA systems usually first require searching for knowledge from the Internet or large-scale databases, then generate the corresponding answers according to the retrieved knowledge. Naturally, given similar questions and corresponding answers as demonstrations which are concatenated into inputs Wang et al. (2022); Li et al. (2023c); Huang et al. (2023a), generators in RAG can learn the pattern between questions and answers and infer what answers should be. For some specific QA tasks where a set of reference documents is given, retrievers in RAG would retrieve the relevant documents for concatenation, and then generators in RAG would read the context then generate the final answers via the self-attention mechanism Guu et al. (2020); Lee et al. (2023); Ram et al. (2023b); Asai et al. (2024), which is similar to solving a reading comprehension problem. Besides, Fabbri et al. Fabbri et al. (2020) focus on designing effective templates for re-organizing the concatenated contexts. Baek et al. Baek et al. (2023) leverage the knowledge graph to retrieve the related facts for the input questions, then feed their concatenation and inputs into the generators. Instead of directly concatenating texts, another branch of works focuses on joining the retrieval embeddings with input embeddings for the encoder-decoder models Izacard and Grave (2021); Sachan et al. (2021); de Jong et al. (2023b); Izacard et al. (2023).

Some works incorporate the external knowledge in the hidden states or the final logits of generators. For the fusion in the hidden states, the key is what kind of knowledge representation would be injected, such as entities Févry et al. (2020); de Jong et al. (2022), chunks Borgeaud et al. (2022); Wang et al. (2023b), documents Cheng et al. (2023a). For the fusion in the logits, most works combine the logits of retrievals and inputs by ensemble techniques Shi et al. (2023); Guu et al. (2020); Lewis et al. (2020); Mueller et al. (2023).

Instead of designing different knowledge fusions for QA systems, existing works also improve QA systems with RAG from other aspects. Some works [Guo et al. \(2019\)](#); [Paranjape et al. \(2022\)](#); [Lin et al. \(2022\)](#) use retrieved question-answering pairs as extra training data. Some works optimize the retriever module, e.g., improving the keys’ representation when building the retriever database [Ram et al. \(2023a\)](#), replacing the indexing with a pre-trained ranking model [Yu et al. \(2023b\)](#), or enabling retrieving phrases with two queries [Min et al. \(2023\)](#). Other works focus on accelerating the generation efficiency of RAG. [Jong et al. de Jong et al. \(2023a\)](#) propose the layer-sparse cross-attention to speed up the decoding. Some works [Asai et al. \(2024\)](#); [Jiang et al. \(2023b\)](#); [Wang et al. \(2023e\)](#) observe that the retrievals may not always provide useful information during the generation process and learn to determine when to retrieve. Moreover, [Sun et al. Sun et al. \(2023\)](#) combine the RAG with agents to iteratively reason the final results.

8.5 Information Extraction

Information Extraction (IE) is a critical task in NLP to automatically extract structured information from unstructured and semi-structured text sources. This task encompasses several sub-tasks, including Named Entity Recognition (NER), Entity Linking (EL), Coreference Resolution (CR), Relation Extraction (RE), etc. The goal is to identify and classify key elements from text and understand the relationships between them, thereby converting textual data into a structured format amenable to analysis and interpretation.

With RAG techniques, addressing IE tasks can be significantly improved in terms of not only performance but also interpretability. In NER tasks, [Wang et al. Wang et al. \(2021b\)](#) first retrieve similar sentences and then concatenate the ranked retrievals for better semantic representations. [Ren et al. Ren et al. \(2023\)](#) show that naive RAG may not address Event Argument Extraction (EAE) tasks. Thus, they adopt a sampling-based method to guarantee the same distribution of event labels between retrievals and inputs then concatenate retrieval texts into inputs for better performance in EAE tasks. Table augmentation is also a challenging task, which requires extracting information from tables. [Glass et al. Glass et al. \(2023\)](#) propose to extract information in a retrieval-augmented manner.

8.6 Text Classification

Text classification tasks are common in NLP applications. Sentiment analysis, a prominent text classification task in NLP, entails identifying and categorizing the emotional tone conveyed in a text. For example, given a sentence of “I love to watch movies”, the analysis models should determine whether it has a positive attitude or a negative attitude. The attitude in sentiment analysis can range from positive to negative or can be neutral, nuanced, and even mixed. The sentiment analysis task is crucial for understanding consumer feedback, monitoring brand reputation, and gaining insights into public opinion on various issues.

RAG techniques can significantly enhance sentiment analysis with different external knowledge fusion strategies. [Li et al. Li et al. \(2023c\)](#) concatenate the retrieved

options and corresponding prompt-based labels with input options. Other works [Chen et al. \(2022b\)](#); [Guo et al. \(2023\)](#) concatenate the retrieval embeddings with input embeddings before feeding them into the decoder. Some works fuse the retrieval features into the hidden states of generators via cross-attention [Cheng et al. \(2023a\)](#); [Wang et al. \(2023d\)](#) or ranking-based addition [Wu et al. \(2024\)](#). Besides, other works focus on fusing the logits of retrievals with the output logit using ensemble techniques [Zhang et al. \(2023b\)](#); [Yu et al. \(2023a\)](#). Except for knowledge fusions, Min et al. [Min et al. \(2023\)](#) enable locating knowledge in phrases more accurately via two queries.

8.7 Dialogue Systems

Dialogue systems, also known as conversational agents or chatbots, are designed to simulate conversation with human users, either in text or speech form. These systems can be categorized into two main types: task-oriented systems [Huang et al. \(2023b\)](#), which assist users in completing specific tasks such as booking tickets or ordering food, and open-domain systems, which aim to carry on a general conversation on a wide range of topics [Shuster et al. \(2021\)](#). The core challenge in developing effective dialogue systems lies in understanding user intent, maintaining context, and generating coherent, relevant responses.

Existing works improve the dialogue system with RAG mostly via the query-based fusions. Some works [Li et al. \(2021\)](#); [King and Flanigan \(2023\)](#); [Cheng et al. \(2023b\)](#) concatenate the retrieved history conversations with current inputs. Other works [Fan et al. \(2021\)](#); [Liu et al. \(2023b\)](#); [Cheng et al. \(2023b\)](#) first leverage an encoder to encode the history responses, then feed the concatenated embeddings into a decoder to generate new responses.

9 RAG Tools

9.1 LLM-based Autonomous Agents

LLM-based autonomous agents are intelligent software systems that leverage the power of LLMs to perform tasks without the need for continuous human intervention [Li et al. \(2024\)](#); [Xi et al. \(2023\)](#); [Wang et al. \(2024\)](#). These agents use LLMs as a brain or controller [Huang et al. \(2024\)](#), and extend their abilities through multimodal perception [Xie et al. \(2024\)](#), tool utilization [Schick et al. \(2023\)](#) and external memory [Packer et al. \(2023\)](#). Especially, external long-term memory for agents functions as the knowledge datastore in RAG, which provides agents with the capability to incorporate external knowledge over extended periods. Therefore, applying RAG would be beneficial to access a broader range of information, improving agents' decision-making and problem-solving abilities [Zhang et al. \(2024b\)](#). This section explores how LLM-based agents can leverage RAG from two perspectives.

Using RAG to Retrieve from External Memory. LLM-based agents can utilize RAG to access and retrieve information from their own external memory [Hatalis et al. \(2023\)](#); [Zhang et al. \(2024c\)](#); [Mei et al. \(2024\)](#). This external memory serves as a knowledge base that the agent can draw upon to enhance its understanding and

decision-making. When faced with a query or a task, the agent can use RAG to retrieve relevant information from this memory, which is then integrated into the generation process of the LLM. This allows the agent to produce responses or solutions that are informed by a wider range of knowledge, leading to more accurate and contextually relevant outcomes.

The ability to tap into a vast external memory enables the agent to continuously learn and adapt based on new information, making it more effective over time. **Using Tools to Search the Web and RAG for Up-to-Date Information.** In addition to retrieving information from its own memory, an LLM-based agent can use tools to search the web for the most current information [Schick et al. \(2023\)](#). This capability is particularly useful for tasks that require up-to-date knowledge, such as news summarization, market analysis, or responding to rapidly evolving situations. Once the agent retrieves the latest information from the web, it can use RAG to integrate this data into its generation process. By combining the LLM’s natural language understanding with real-time data from the web, the agent can generate responses that are not only contextually relevant but also reflect the latest developments. This approach enhances the agent’s ability to provide accurate and timely information, improving its effectiveness in dynamic environments.

In both cases, RAG plays a crucial role in augmenting the capabilities of LLM-based agents by enabling them to access and leverage a wider range of information, whether it’s from their own external memory or from real-time sources on the web. This leads to more informed decision-making and enhances the overall performance of the agents.

9.2 Frameworks

Frameworks like Langchain [LangChain \(2023\)](#) and LLaMAindex [Liu \(2022\)](#) pose a significant impact on enhancing the practical implementation of RAG. Langchain and LLaMAindex exemplify the integration of sophisticated retrieval mechanisms with generative models, facilitating the seamless incorporation of external data into the language generation process. This section will introduce these two representative RAG frameworks in details.

Langchain is a framework designed to augment the capabilities of language models by integrating them with external knowledge sources and databases. It acts as a middleware that facilitates the interaction between language models and various data retrieval systems, enabling a more informed and accurate generation of responses. The core functionality of Langchain involves orchestrating the flow of information from external databases into the generative process of language models, enhancing their ability to leverage context and specific knowledge in their responses. This integration plays a crucial role in enabling language models to perform tasks that require access to up-to-date or detailed information that is not contained within the model’s initial training data.

LLaMAindex is a specialized data framework that focuses on organizing and indexing vast amounts of data to improve the retrieval capabilities of language models. This framework supports efficient querying mechanisms, allowing language models to quickly access relevant information from a structured repository. LLaMAindex is

designed to be highly scalable and can handle diverse data types, from text documents to structured databases. The indexed data supports a wide range of applications, from simple fact retrieval to complex analytical tasks, making it an indispensable tool for enhancing the information retrieval phase in language models.

Both Langchain and LLaMAindex are deeply connected to the concept of RAG. Langchain enhances RAG by providing a structured way for language models to interact with external databases and knowledge sources during the generation process. On the other hand, LLaMAindex serves as a powerful backend for RAG systems by ensuring that the retrieval process is both fast and relevant. Together, Langchain and LLaMAindex enhance the capabilities of RAG by ensuring that the language models are not only generating text based on their internal knowledge but are also capable of pulling in external data to provide responses that are contextually enriched and informationally robust.

10 Discussion and Future Direction

Despite the success of the RAG for natural language processing, some challenges should be considered. This paper highlights these challenges to inspire future research and provides possible future research directions in RAG for NLP.

10.1 Retrieval Quality

The retrieval quality largely determines the performance of the generator. Park et al. [Park and Lee \(2024\)](#) investigated imperfect retrieval’s effect and demonstrated that unanswerability or contradiction of a document set, which frequently leads to hallucinations. However, Ren et al. [Ren et al. \(2025\)](#) demonstrated that the improvement brought about by the increase in the number of supporting documents is not entirely due to the increase in recall. To this end, using various evaluation methods referred to in Section 7 for retrieval information will be more beneficial to model generation. The retrieval quality involves the following four key factors that must be designed. The first consideration is **determining the optimal key** to use in the vector database. This process typically involves subjective decision-making and requires human effort to design effectively. The naive idea is to choose inputs for the given tasks, treating each task as a QA problem.

The second is **the choice of embedding model**. After determining the key, the next step is leveraging embedding models to convert text into vector representations. Models such as BERT [Devlin et al. \(2019\)](#), RoBERTa [Liu et al. \(2019\)](#), or domain-specific embeddings can be crucial to determine how well nuances and contextual meanings are captured. Adapting the embedding model to suit specific data types or queries better can significantly enhance retrieval quality. This requires training the model on domain-specific corpora, including the types of queries and documents the system will encounter.

Thirdly, **designing effective similarity metrics** is also crucial to improve retrieval quality. The goal of similarity metrics is to measure the relevance between the query and the retrieved information. Some classical similarity metrics, such as cosine similarity or Euclidean distance, used for ranking in the recommender system

can also be used in RAG [Gunawardana and Shani \(2009\)](#). Apart from these metrics, some works explored more complex similarity metrics, such as optimal transport distance [Cui et al. \(2023\)](#), to obtain a task-specific similarity.

Finally, **approximate nearest neighbor (ANN) searching** is also a key step in determining what knowledge should be returned as nearest neighbors. Advanced ANN searching aims to accelerate the retrieval efficiency at the cost of sacrificing the retrieval quality. Choosing a suitable ANN algorithm, such as product quantization [Jégou et al. \(2011\)](#) or HNSW [Malkov and Yashunin \(2020\)](#), requires a good trade-off between retrieval efficiency and retrieval quality. All of these factors collectively contribute to the retrieval quality of the retriever.

10.2 RAG Efficiency

RAG efficiency is crucial for downstream NLP applications, which limits the volume of data that can be retrieved. There are two simple ways to guarantee RAG efficiency without new algorithms, i.e., reducing the volume of data or adding more powerful computing and memory resources. However, the former may impact the retrieval quality, while the latter requires more resource cost.

RAG efficiency encompasses the efficiency of the retriever and the efficiency of retrieval fusions. Retriever efficiency refers to the time cost of retrieving relevant information, which can be divided into three parts, i.e., encoding time, ANN searching time, and data fetching time of the datastore. It is unnecessary to jointly optimize all three components as the bottleneck would vary from different database sizes. For smaller retrieval databases, such as those with fewer than 1 million entries, the encoding phase is often the primary bottleneck, as the vector database can be all stored in the memory. Several topics, such as model quantization [Kim et al. \(2021\)](#); [Bai et al. \(2021\)](#), distillation [Jiao et al. \(2020\)](#); [Ding et al. \(2023\)](#), or model pruning [Ganesh et al. \(2021\)](#), are used to accelerate the encoding.

In contrast, for larger databases, the time cost of searching in the index and fetching data from the datastore becomes the major bottleneck, as the searching is over a considerable amount of data, and the fetching involves I/O overheads. In this case, efficient ANN searching algorithms [Johnson et al. \(2021\)](#); [Douze et al. \(2024\)](#); [Guo et al. \(2020\)](#) and system-level optimizations [Jin et al. \(2024\)](#); [Jiang et al. \(2024\)](#) are the main focus.

Retrieval fusion efficiency, which aims to enhance the inference efficiency when integrating retrievals, is worth to be optimized for improving the RAG efficiency. For example, the computational overhead of query-based fusion is often non-negligible due to the long sequence length. Some works, such as Fid-light [Hofstätter et al. \(2023\)](#) and ReFusion [Wu et al. \(2024\)](#), mainly target reducing the computations while integrating the retrieved information.

10.3 Discussion for Query-based Fusion in RAG

Two main query-based fusion RAG concerns must be discussed. The first is the performance comparison towards long-context LLMs. Hui et al. [Hui et al. \(2024\)](#) demonstrated that RAG system and long-context LLMs show similar performance for

free-form or knowledge-based tasks (e.g., paper-based and wiki-based Q&A). However, in tasks requiring numerical reasoning (e.g., financial Q&A), long-context LLMs underperform compared to RAG, as the verbose content in long contexts can obscure key facts and hinder precise reasoning. Additionally, RAG tends to outperform long-context mechanisms in smaller LLMs, which struggle to process large volumes of data effectively. The second concern is improving the query-based fusion method, which is prevalent in modern RAG systems. Recent works used prompt engineering strategies [Lewis et al. \(2020\)](#); [Guo et al. \(2023\)](#), query refinement techniques [Yan et al. \(2024\)](#), calibration-based strategies [Jiang et al. \(2023b\)](#); [Asai et al. \(2024\)](#), and iterative RAG [Shao et al. \(2023\)](#) to improve the query-based fusion RAG. Among these strategies, prompt engineering and calibration offer improved results with minimal overhead, while iterative retrieval-augmented generation (RAG) and query refinement prioritize higher computational costs in exchange for potential accuracy gains. Prompt engineering and query refinement depend on manual tuning, whereas calibration and iterative RAG are more automation-friendly but require high-quality, robust data. Simpler strategies, such as prompting, are easier to deploy but lack flexibility, while more complex methods like iterative RAG provide greater adaptability at the expense of stability. These approaches can be combined (e.g., refined queries with iterative loops and calibrated prompts), though such synergies must carefully balance the compounded drawbacks, such as increased latency and complexity.

10.4 Choices of Fusions

This paper introduces three kinds of retrieval fusions, where each fusion is worth further exploring. Query-based fusions concatenate the texts or embeddings of retrieved knowledge with inputs. These methods have better interpretability and are easy to apply even only when the API of LLMs is provided. However, concatenation leads to a long sequence of inputs, thus resulting in a large computational overhead in the attention and truncation of inputs. Some works [Wu et al. \(2024\)](#); [Areifeen et al. \(2023\)](#) aim to improve efficiency when integrating retrievals, while others [Bertsch et al. \(2023\)](#); [Wang et al. \(2023d\)](#) focus on improving the efficiency when increasing the model input length.

Conversely, latent-based fusions amalgamate information at a deeper, more abstract level, which may capture more nuanced relationships between the retrieved information and the query. However, these fusions significantly lack interpretability and often require pre-training or fine-tuning to adjust the retrieval embeddings or reweight the retrievals. Therefore, enhancing the interpretability of such latent-based fusions is also worth exploring in the future.

Logits-based fusions incorporate information at the decision level, thereby offering a potentially more flexible and robust integration of data from various sources. Nonetheless, these fusions may oversimplify the fusion process, diminishing the richness of the retrieved information by reducing them to logit values. Meanwhile, such fusions require performing all inference of retrievals, which is also a time-consuming process.

Apart from applying one kind of fusion in practical applications, combining different fusions is also worth exploring for better performance. These fusion methods are

not mutually exclusive, as they focus on augmenting the different stages of generators, i.e., inputs, hidden states, and outputs. Besides, during the generation, when to fuse retrieved knowledge is also a significant problem worthy of further exploration [Mallen et al. \(2023\)](#).

10.5 RAG Training

As introduced in Section 6, RAG training includes two branches of works, RAG with/without datastore update. For RAG without a datastore update, the main challenge is how to jointly optimize all parameters in RAG. This may involve new loss functions with multiple objectives, new optimizations for efficient tuning parameters in the retriever and generator, or other training strategies.

For RAG with datastore update, one challenge is how to align the retrieval representations with the generator’s representations. Although the time cost of the update operation in datastore cannot be ignored, some works [Chen et al. \(2022b\)](#) reduce the update frequency by asynchronously updating, thus achieving the alignment of knowledge representation and model’s representation. Another challenge is when to retrain/fine-tune the generator in RAG when a new corpus is added. Due to the in-context learning capability of existing LLM-based generators and high training overhead, retraining/fine-tuning the generator or directly inferring the generator becomes a challenging choice for different scenarios. Recently, some efficient training strategies [Hu et al. \(2022\)](#); [Dettmers et al. \(2023\)](#) have been proposed to accelerate the fine-tuning process, which can be taken into consideration.

10.6 Cross-Modality Retrieval

Retrieving cross-modality information in NLP tasks can greatly enhance the quality and richness of the representations, leading to improved performance. First, cross-modality information, such as combining text with images, videos, or audio, provides a richer context to the content [Hu \(2023\)](#). For instance, when language is ambiguous, accompanying images can clarify meanings difficult to convey through text alone. Second, different modalities can contribute various types of information that are not accessible from a single source. For example, visual data can provide spatial, color, and action cues, while textual data can offer detailed descriptions, emotions, or abstract concepts. Combining these can lead to a more comprehensive understanding of the data. Moreover, Models trained on multi-modal data typically exhibit increased robustness and generalizability [Wu and Goodman \(2018\)](#). These models are adept at associating information across diverse inputs, mitigating overfitting to the peculiarities of a single modality. This attribute is particularly valuable in real-world applications of NLP, such as in autonomous vehicles, where systems must interpret textual information from signs or dialogues and sensory data from the surrounding environment to make informed decisions. Furthermore, multi-modal data can resolve ambiguities that cannot be resolved within a single modality. For example, the phrase “bank” can refer to either a financial institution or the side of a river, and visual context can help disambiguate this. Last, human communication is inherently multi-modal, incorporating elements such as gestures, facial expressions, and tone of voice. Systems capable

of processing multiple modes of communication can interact with humans in a manner that is both more natural and intuitive. In conclusion, integrating cross-modality information in RAG for NLP tasks not only enhances the richness and quality of data representations but also significantly improves the systems’ comprehension, interaction capabilities, and adaptability to diverse applications.

11 Conclusion

In this survey, we delve into the development of RAG within the field of natural language processing. First, this paper introduces the components of RAG and their functionalities. Subsequently, this paper elaborates on each step involved in retriever, discussing the diverse techniques. Furthermore, this paper categorizes the retrieval fusions, evaluating the strengths and weaknesses inherent in each retrieval fusion technique. Besides, this paper discusses the RAG training, including RAG with/without datastore update. Then, this paper presents RAG evaluation and benchmarking, and explores how RAG can be adapted for various NLP tasks and provides practical applications of RAG in real-world scenarios. Conclusively, this paper identifies ongoing challenges and suggests directions for future research to foster advancements in this evolving area.

12 Statements and Declarations

12.1 Authors’ Contribution

S.W. and Y.X., wrote the main manuscript text. Y.C., H.W., C.C., and Y.Y. wrote and refined several sections. S.W., Y.X., Y.C., H.W., C.C., Y.Y, L.H, X.L, T.K, N.G, and C.X participated in the main investigation and the design of the paper’s structure. All authors participated in designing and refining the methodology. All authors reviewed the manuscript.

12.2 Conflict of Interest

The authors declare no Conflict of interest.

References

- Abnar S, Dehghani M, Neyshabur B, et al (2022) Exploring the limits of large scale pre-training. In: The Tenth International Conference on Learning Representations (ICLR)
- Adlakha V, BehnamGhader P, Lu XH, et al (2024) Evaluating correctness and faithfulness of instruction-following models for question answering. *Trans Assoc Comput Linguistics* 12:681–699
- Ainslie J, Lee-Thorp J, de Jong M, et al (2023) GQA: training generalized multi-query transformer models from multi-head checkpoints. In: Bouamor H, Pino J,

- Bali K (eds) Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP). Association for Computational Linguistics, pp 4895–4901
- AlKhamissi B, Li M, Celikyilmaz A, et al (2022) A review on language models as knowledge bases. CoRR abs/2204.06031
- Anil R, Borgeaud S, Wu Y, et al (2023) Gemini: A family of highly capable multimodal models. CoRR abs/2312.11805
- Arefeen MA, Debnath B, Chakradhar S (2023) Leancontext: Cost-efficient domain-specific question answering using llms. CoRR abs/2309.00841
- Asai A, Wu Z, Wang Y, et al (2024) Self-rag: Learning to retrieve, generate, and critique through self-reflection. In: The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net
- Baek J, Aji AF, Saffari A (2023) Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. CoRR abs/2306.04136
- Bai H, Zhang W, Hou L, et al (2021) Binarybert: Pushing the limit of BERT quantization. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL/IJCNLP). Association for Computational Linguistics, pp 4334–4348
- Bertsch A, Alon U, Neubig G, et al (2023) Unlimiformer: Long-range transformers with unlimited length input. In: Advances in Neural Information Processing Systems 36 (NeurIPS)
- Borgeaud S, Mensch A, Hoffmann J, et al (2022) Improving language models by retrieving from trillions of tokens. In: Proceedings of the 39th International Conference on Machine Learning (ICML), pp 2206–2240
- Brown TB, Mann B, Ryder N, et al (2020) Language models are few-shot learners. In: Advances in Neural Information Processing Systems 33 (NeurIPS)
- Cai D, Wang Y, Li H, et al (2021) Neural machine translation with monolingual translation memory. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL/IJCNLP), pp 7307–7318
- Chen J, Chen Q, Li D, et al (2022a) Sedr: Segment representation learning for long documents dense retrieval. CoRR abs/2211.10841
- Chen J, Lin H, Han X, et al (2024) Benchmarking large language models in retrieval-augmented generation. In: Wooldridge MJ, Dy JG, Natarajan S (eds) Thirty-Eighth

- AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20-27, 2024, Vancouver, Canada. AAAI Press, pp 17754–17762
- Chen X, Li L, Zhang N, et al (2022b) Decoupling knowledge from memorization: Retrieval-augmented prompt learning. In: Advances in Neural Information Processing Systems 35 (NeurIPS)
- Cheng X, Lin Y, Chen X, et al (2023a) Decouple knowledge from parameters for plug-and-play language modeling. In: Findings of the Association for Computational Linguistics (ACL), pp 14288–14308
- Cheng X, Luo D, Chen X, et al (2023b) Lift yourself up: Retrieval-augmented text generation with self-memory. In: Advances in Neural Information Processing Systems 36 (NeurIPS)
- Chevalier A, Wettig A, Ajith A, et al (2023) Adapting language models to compress contexts. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 3829–3846
- Chuang Y, Fang W, Li S, et al (2023) Expand, rerank, and retrieve: Query reranking for open-domain question answering. In: Findings of the Association for Computational Linguistics (ACL). Association for Computational Linguistics, pp 12131–12147
- Clark K, Luong M, Le QV, et al (2020) ELECTRA: pre-training text encoders as discriminators rather than generators. In: 8th International Conference on Learning Representations (ICLR). OpenReview.net
- Colombo P, Pires TP, Boudiaf M, et al (2024) Saullm-7b: A pioneering large language model for law. CoRR abs/2403.03883
- Cui Y, Liu Z, Chen Y, et al (2023) Retrieval-augmented multiple instance learning. In: Advances in Neural Information Processing Systems 36 (NeurIPS)
- Dai Y, Zhang Z, Liu Q, et al (2023) Simple and scalable nearest neighbor machine translation. In: The Eleventh International Conference on Learning Representations (ICLR)
- Dale D, Voita E, Barrault L, et al (2023) Detecting and mitigating hallucinations in machine translation: Model internal workings alone do well, sentence similarity even better. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL). Association for Computational Linguistics, pp 36–50

- Deguchi H, Watanabe T, Matsui Y, et al (2023) Subset retrieval nearest neighbor machine translation. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 174–189
- Dettmers T, Pagnoni A, Holtzman A, et al (2023) Qlora: Efficient finetuning of quantized llms. In: Advances in Neural Information Processing Systems 36 (NeurIPS)
- Devlin J, Chang M, Lee K, et al (2019) BERT: pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), pp 4171–4186
- Ding Y, Fan W, Ning L, et al (2024) A survey on RAG meets llms: Towards retrieval-augmented large language models. CoRR abs/2405.06211
- Ding Z, Jiang G, Zhang S, et al (2023) SKDBERT: compressing BERT via stochastic knowledge distillation. In: Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI). AAAI Press, pp 7414–7422
- Doostmohammadi E, Norlund T, Kuhlmann M, et al (2023) Surface-based retrieval reduces perplexity of retrieval-augmented language models. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 521–529
- Douze M, Guzhva A, Deng C, et al (2024) The faiss library. CoRR abs/2401.08281
- explosion (2016) Spacy. misc, URL <https://spacy.io/>
- Fabbri AR, Ng P, Wang Z, et al (2020) Template-based question generation from retrieved sentences for improved unsupervised question answering. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp 4508–4513
- Facebook (2013) RocksDB. Software, URL <https://github.com/facebook/rocksdb>
- Fan A, Gardent C (2022) Generating full length wikipedia biographies: The impact of gender bias on the retrieval-based generation of women biographies. CoRR abs/2204.05879
- Fan A, Gardent C, Braud C, et al (2021) Augmenting transformers with knn-based composite memory for dialog. Trans Assoc Comput Linguistics 9:82–99
- Férvy T, Soares LB, FitzGerald N, et al (2020) Entities as experts: Sparse memory access with entity supervision. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 4937–4951

- Friel R, Belyi M, Sanyal A (2024) Ragbench: Explainable benchmark for retrieval-augmented generation systems. arXiv preprint arXiv:2407.11005
- Fu Y, Chen C, Chen X, et al (2024) Optimizing the number of clusters for billion-scale quantization-based nearest neighbor search. *IEEE Trans Knowl Data Eng* 36(11):6786–6800
- Ganesh P, Chen Y, Lou X, et al (2021) Compressing large-scale transformer-based models: A case study on BERT. *Trans Assoc Comput Linguistics* 9:1061–1080
- Gao T, Yao X, Chen D (2021) Simcse: Simple contrastive learning of sentence embeddings. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, pp 6894–6910
- Gao Y, Xiong Y, Gao X, et al (2023) Retrieval-augmented generation for large language models: A survey. *CoRR* abs/2312.10997
- Glass MR, Wu X, Naik AR, et al (2023) Retrieval-based transformer for table augmentation. In: *Findings of the Association for Computational Linguistics (ACL)*, pp 5635–5648
- Gong H, Shen Y, Yu D, et al (2020) Recurrent chunking mechanisms for long-text machine reading comprehension. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, pp 6751–6761
- Gunawardana A, Shani G (2009) A survey of accuracy evaluation metrics of recommendation tasks. *J Mach Learn Res* 10:2935–2962
- Guo D, Tang D, Duan N, et al (2019) Coupling retrieval and meta-learning for context-dependent semantic parsing. In: *Proceedings of the 57th Conference of the Association for Computational Linguistics (ACL)*, pp 855–866
- Guo D, Yang D, Zhang H, et al (2025) Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948
- Guo R, Sun P, Lindgren E, et al (2020) Accelerating large-scale inference with anisotropic vector quantization. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, *Proceedings of Machine Learning Research*, vol 119. PMLR, pp 3887–3896
- Guo R, Luan X, Xiang L, et al (2022) Manu: A cloud native vector database management system. *Proc VLDB Endow* 15(12):3548–3561
- Guo Z, Cheng S, Wang Y, et al (2023) Prompt-guided retrieval augmentation for non-knowledge-intensive tasks. In: *Findings of the Association for Computational*

- Linguistics (ACL), pp 10896–10912
- Guu K, Lee K, Tung Z, et al (2020) Retrieval augmented language model pre-training. In: Proceedings of the 37th International Conference on Machine Learning (ICML), pp 3929–3938
- Harris D, Harris S (2010) Digital design and computer architecture. Morgan Kaufmann
- Harris ZS (1954) Distributional structure. *Word* 10(2-3):146–162
- Hatalis K, Christou D, Myers J, et al (2023) Memory matters: The need to improve long-term memory in llm-agents. In: Proceedings of the AAAI Symposium Series, pp 277–280
- He Q, Wang Y, Wang W (2024) Can language models act as knowledge bases at scale? CoRR abs/2402.14273
- Hoffmann J, Borgeaud S, Mensch A, et al (2022) Training compute-optimal large language models. CoRR abs/2203.15556
- Hofstätter S, Chen J, Raman K, et al (2023) Fid-light: Efficient and effective retrieval-augmented text generation. In: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR). ACM, pp 1437–1447
- Hossain N, Ghazvininejad M, Zettlemoyer L (2020) Simple and effective retrieve-edit-rerank text generation. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp 2532–2538
- Hu EJ, Shen Y, Wallis P, et al (2022) Lora: Low-rank adaptation of large language models. In: The Tenth International Conference on Learning Representations (ICLR)
- Hu X (2023) Multimodal named entity recognition and relation extraction with retrieval-augmented strategy. In: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR). ACM, p 3488
- Hu Y, Lu Y (2024) RAG and RAU: A survey on retrieval-augmented language model in natural language processing. CoRR abs/2404.19543
- Huang J, Ping W, Xu P, et al (2023a) RAVEN: in-context learning with retrieval augmented encoder-decoder language models. CoRR abs/2308.07922
- Huang Q, Tung AKH (2023) Lightweight-yet-efficient: Revitalizing ball-tree for point-to-hyperplane nearest neighbor search. In: 39th IEEE International Conference on Data Engineering (ICDE). IEEE, pp 436–449

- Huang Q, Fu S, Liu X, et al (2023b) Learning retrieval augmentation for personalized dialogue generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP). Association for Computational Linguistics, pp 2523–2540
- Huang X, Liu W, Chen X, et al (2024) Understanding the planning of LLM agents: A survey. CoRR abs/2402.02716
- Huang Y, Huang J (2024) A survey on retrieval-augmented text generation for large language models. CoRR abs/2404.10981
- Huang Y, Liu D, Zhong Z, et al (2023c) knn-adapter: Efficient domain adaptation for black-box language models. CoRR abs/2302.10879
- Hui Y, Lu Y, Zhang H (2024) UDA: A benchmark suite for retrieval augmented generation in real-world document analysis. In: Globersons A, Mackey L, Belgrave D, et al (eds) Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024
- Ishiwatari S, Yao J, Liu S, et al (2017) Chunk-based decoder for neural machine translation. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL). Association for Computational Linguistics, pp 1901–1912
- Izacard G, Grave E (2021) Leveraging passage retrieval with generative models for open domain question answering. In: Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL), pp 874–880
- Izacard G, Caron M, Hosseini L, et al (2022) Unsupervised dense information retrieval with contrastive learning. Trans Mach Learn Res 2022
- Izacard G, Lewis PSH, Lomeli M, et al (2023) Atlas: Few-shot learning with retrieval augmented language models. J Mach Learn Res 24:251:1–251:43
- Jégou H, Douze M, Schmid C (2011) Product quantization for nearest neighbor search. IEEE Trans Pattern Anal Mach Intell 33(1):117–128
- Ji Z, Lee N, Frieske R, et al (2023a) Survey of hallucination in natural language generation. ACM Comput Surv 55(12):248:1–248:38
- Ji Z, Liu Z, Lee N, et al (2023b) RHO: reducing hallucination in open-domain dialogues with knowledge grounding. In: Findings of the Association for Computational Linguistics (ACL). Association for Computational Linguistics, pp 4504–4522
- Jiang AQ, Sablayrolles A, Mensch A, et al (2023a) Mistral 7b. CoRR abs/2310.06825

- Jiang H, Lu Z, Meng F, et al (2022) Towards robust k-nearest-neighbor machine translation. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 5468–5477
- Jiang W, Zhang S, Han B, et al (2024) Piperag: Fast retrieval-augmented generation via algorithm-system co-design. CoRR abs/2403.05676
- Jiang Z, Xu FF, Gao L, et al (2023b) Active retrieval augmented generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 7969–7992
- Jiao X, Yin Y, Shang L, et al (2020) Tinybert: Distilling BERT for natural language understanding. In: Findings of the Association for Computational Linguistics (EMNLP), Findings of ACL, vol EMNLP 2020. Association for Computational Linguistics, pp 4163–4174
- Jin C, Zhang Z, Jiang X, et al (2024) Ragcache: Efficient knowledge caching for retrieval-augmented generation. CoRR abs/2404.12457
- Jin Q, Kim W, Chen Q, et al (2023) Medcpt: Contrastive pre-trained transformers with large-scale pubmed search logs for zero-shot biomedical information retrieval. Bioinform 39(10)
- Johnson J, Douze M, Jégou H (2021) Billion-scale similarity search with gpus. IEEE Trans Big Data 7(3):535–547
- de Jong M, Zemlyanskiy Y, FitzGerald N, et al (2022) Mention memory: incorporating textual knowledge into transformers through entity mention attention. In: The Tenth International Conference on Learning Representations (ICLR)
- de Jong M, Zemlyanskiy Y, Ainslie J, et al (2023a) Fido: Fusion-in-decoder optimized for stronger performance and faster inference. In: Findings of the Association for Computational Linguistics (ACL), pp 11534–11547
- de Jong M, Zemlyanskiy Y, FitzGerald N, et al (2023b) Pre-computed memory or on-the-fly encoding? A hybrid approach to retrieval augmentation makes the most of your compute. In: Proceedings of the 40th International Conference on Machine Learning (ICML), pp 7329–7342
- Kaplan J, McCandlish S, Henighan T, et al (2020) Scaling laws for neural language models. CoRR abs/2001.08361
- Karpukhin V, Oguz B, Min S, et al (2020) Dense passage retrieval for open-domain question answering. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 6769–6781

- Khandelwal U, Fan A, Jurafsky D, et al (2020a) Nearest neighbor machine translation. CoRR abs/2010.00710
- Khandelwal U, Levy O, Jurafsky D, et al (2020b) Generalization through memorization: Nearest neighbor language models. In: The 8th International Conference on Learning Representations (ICLR)
- Kim S, Gholami A, Yao Z, et al (2021) I-BERT: integer-only BERT quantization. In: Proceedings of the 38th International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, vol 139. PMLR, pp 5506–5518
- King B, Flanigan J (2023) Diverse retrieval-augmented in-context learning for dialogue state tracking. In: Findings of the Association for Computational Linguistics (ACL), pp 5570–5585
- LangChain (2023) LangChain. Software, URL <https://www.langchain.com/>
- Lazaridou A, Gribovskaya E, Stokowiec W, et al (2022) Internet-augmented language models through few-shot prompting for open-domain question answering. CoRR abs/2203.05115
- Lee K, Han S, Hwang S, et al (2023) When to read documents or QA history: On unified and selective open-domain QA. In: Findings of the Association for Computational Linguistics (ACL), pp 6420–6432
- Lee S, Shakir A, Koenig D, et al (2024) Open source strikes bread - new fluffy embeddings model. URL <https://www.mixedbread.ai/blog/mxbai-embed-large-v1>
- Lewis PSH, Perez E, Piktus A, et al (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems 33 (NeurIPS)
- Lewis PSH, Oguz B, Xiong W, et al (2022) Boosted dense retriever. In: Carpuat M, de Marneffe M, Ruíz IVM (eds) Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2022, Seattle, WA, United States, July 10-15, 2022. Association for Computational Linguistics, pp 3102–3117
- Li H, Su Y, Cai D, et al (2022a) A survey on retrieval-augmented text generation. CoRR abs/2202.01110
- Li M, Wang Y, Zhang P, et al (2023a) Deep learning for approximate nearest neighbour search: A survey and future directions. IEEE Trans Knowl Data Eng 35(9):8997–9018
- Li X, Li P, Hu P (2023b) Revisiting source context in nearest neighbor machine translation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural

- Li X, Lv K, Yan H, et al (2023c) Unified demonstration retriever for in-context learning. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 4644–4668
- Li Y, Yang Y, Quan X, et al (2021) Retrieve & memorize: Dialog policy learning with multi-action memory. In: Findings of the Association for Computational Linguistics (ACL/IJCNLP), pp 447–459
- Li Y, Wen H, Wang W, et al (2024) Personal LLM agents: Insights and survey about the capability, efficiency and security. CoRR abs/2401.05459
- Li Z, Guo R, Kumar S (2022b) Decoupled context processing for context augmented language modeling. In: Advances in Neural Information Processing Systems 35 (NeurIPS)
- Lin BY, Tan K, Miller C, et al (2022) Unsupervised cross-task generalization via retrieval augmentation. In: Advances in Neural Information Processing Systems 35 (NeurIPS)
- Liu J (2022) LlamaIndex. <https://doi.org/10.5281/zenodo.1234>, URL https://github.com/jerryliu/llama_index
- Liu J, Li L, Xiang T, et al (2023a) TCRA-LLM: token compression retrieval augmented large language model for inference cost reduction. In: Findings of the Association for Computational Linguistics (EMNLP), pp 9796–9810
- Liu S, Wei Y (2015) Fast nearest neighbor searching based on improved vp-tree. Pattern Recognit Lett 60-61:8–15
- Liu S, Cho H, Freedman M, et al (2023b) RECAP: retrieval-enhanced context-aware prefix encoder for personalized dialogue response generation. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 8404–8419
- Liu Y, Ott M, Goyal N, et al (2019) Roberta: A robustly optimized BERT pretraining approach. CoRR abs/1907.11692
- LMDB (2014) LMDB. Software, URL <https://github.com/LMDB/lmdb>
- Luo M, Jain S, Gupta A, et al (2023) A study on the efficiency and generalization of light hybrid retrievers. In: Rogers A, Boyd-Graber JL, Okazaki N (eds) Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), ACL 2023, Toronto, Canada, July 9-14, 2023. Association for Computational Linguistics, pp 1617–1626

- Lyu X, Grafberger S, Biegel S, et al (2023) Improving retrieval-augmented large language models via data importance learning. CoRR abs/2307.03027
- Malkov YA, Yashunin DA (2020) Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Trans Pattern Anal Mach Intell 42(4):824–836
- Mallen A, Asai A, Zhong V, et al (2023) When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 9802–9822
- Martins PH, Marinho Z, Martins AFT (2022) Chunk-based nearest neighbor machine translation. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 4228–4245
- Mei K, Li Z, Xu S, et al (2024) AIOS: LLM agent operating system. CoRR abs/2403.16971
- Meng K, Bau D, Andonian A, et al (2022a) Locating and editing factual associations in GPT. In: Advances in Neural Information Processing Systems 35 (NeurIPS)
- Meng Y, Li X, Zheng X, et al (2022b) Fast nearest neighbor machine translation. In: Findings of the Association for Computational Linguistics (ACL), pp 555–565
- Mesnard T, Hardin C, Dadashi R, et al (2024) Gemma: Open models based on gemini research and technology. CoRR abs/2403.08295
- Min S, Shi W, Lewis M, et al (2023) Nonparametric masked language modeling. In: Findings of the Association for Computational Linguistics (ACL), pp 2097–2118
- Mueller A, Narang K, Mathias L, et al (2023) Meta-training with demonstration retrieval for efficient few-shot learning. In: Findings of the Association for Computational Linguistics (ACL), pp 6049–6064
- Muszynska E (2016) Graph- and surface-level sentence chunking. In: Proceedings of the ACL 2016 Student Research Workshop. Association for Computational Linguistics, pp 93–99
- NLTK (2001) NLTK. misc, URL <https://www.nltk.org/>
- OpenAI (2022) Text-Emb-Ada. URL <https://platform.openai.com/docs/guides/embeddings>
- OpenAI (2023) GPT-4 technical report. CoRR abs/2303.08774
- Packer C, Fang V, Patil SG, et al (2023) Memgpt: Towards llms as operating systems. CoRR abs/2310.08560

- Paranjape B, Lamm M, Tenney I (2022) Retrieval-guided counterfactual generation for QA. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL), pp 1670–1686
- Park S, Lee J (2024) Toward robust ralms: Revealing the impact of imperfect retrieval on retrieval-augmented language models. *Trans Assoc Comput Linguistics* 12:1686–1702
- Petroni F, Rocktäschel T, Riedel S, et al (2019) Language models as knowledge bases? In: Inui K, Jiang J, Ng V, et al (eds) Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). Association for Computational Linguistics, pp 2463–2473
- Pipitone N, Alami GH (2024) Legalbench-rag: A benchmark for retrieval-augmented generation in the legal domain. *CoRR* abs/2408.10343
- Radford A, Narasimhan K, Salimans T, et al (2018) Improving language understanding by generative pre-training. OpenAI blog
- Radford A, Wu J, Child R, et al (2019) Language models are unsupervised multitask learners. OpenAI blog 1(8):9
- Rajaraman A, Ullman JD (2011) *Data Mining*, Cambridge University Press, p 1–17
- Ram O, Bezael L, Zicher A, et al (2023a) What are you token about? dense retrieval as distributions over the vocabulary. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 2481–2498
- Ram O, Levine Y, Dalmedigos I, et al (2023b) In-context retrieval-augmented language models. *Trans Assoc Comput Linguistics* 11:1316–1331
- Ram P, Sinha K (2019) Revisiting kd-tree for nearest neighbor search. In: Teredesai A, Kumar V, Li Y, et al (eds) Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD). ACM, pp 1378–1388
- Reid M, Savinov N, Teplyashin D, et al (2024) Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *CoRR* abs/2403.05530
- Reimers N, Gurevych I (2019) Sentence-bert: Sentence embeddings using siamese bert-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp 3980–3990
- Ren R, Wang Y, Qu Y, et al (2025) Investigating the factual knowledge boundary of large language models with retrieval augmentation. In: Rambow O, Wanner L, Apidianaki M, et al (eds) Proceedings of the 31st International Conference on

- Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025. Association for Computational Linguistics, pp 3697–3715
- Ren Y, Cao Y, Guo P, et al (2023) Retrieve-and-sample: Document-level event argument extraction via hybrid retrieval augmentation. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 293–306
- Robertson SE, Zaragoza H (2009) The probabilistic relevance framework: BM25 and beyond. *Found Trends Inf Retr* 3(4):333–389
- Saad-Falcon J, Khattab O, Potts C, et al (2024) ARES: an automated evaluation framework for retrieval-augmented generation systems. In: Duh K, Gómez-Adorno H, Bethard S (eds) Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024. Association for Computational Linguistics, pp 338–354
- Sachan DS, Reddy S, Hamilton WL, et al (2021) End-to-end training of multi-document reader and retriever for open-domain question answering. In: Advances in Neural Information Processing Systems 34 (NeurIPS), pp 25968–25981
- Sanh V, Debut L, Chaumond J, et al (2019) Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter. *CoRR* abs/1910.01108
- Schick T, Dwivedi-Yu J, Dessì R, et al (2023) Toolformer: Language models can teach themselves to use tools. In: Advances in Neural Information Processing Systems 36 (NeurIPS)
- Shao Z, Gong Y, Shen Y, et al (2023) Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. In: Bouamor H, Pino J, Bali K (eds) Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023. Association for Computational Linguistics, pp 9248–9274
- Shi W, Min S, Yasunaga M, et al (2023) REPLUG: retrieval-augmented black-box language models. *CoRR* abs/2301.12652
- Shuster K, Poff S, Chen M, et al (2021) Retrieval augmentation reduces hallucination in conversation. In: Findings of the Association for Computational Linguistics (EMNLP). Association for Computational Linguistics, pp 3784–3803
- Singhal K, Azizi S, Tu T, et al (2023a) Large language models encode clinical knowledge. *Nature* 620(7972):172–180
- Singhal K, Tu T, Gottweis J, et al (2023b) Towards expert-level medical question answering with large language models. *CoRR* abs/2305.09617

- Spotify (2017) Annoy. Software, URL <https://github.com/spotify/annoy>
- Su J, Ahmed MHM, Lu Y, et al (2024) Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568:127063
- Sun J, Xu C, Tang L, et al (2023) Think-on-graph: Deep and responsible reasoning of large language model with knowledge graph. *CoRR* abs/2307.07697
- Touvron H, Lavril T, Izacard G, et al (2023a) Llama: Open and efficient foundation language models. *CoRR* abs/2302.13971
- Touvron H, Martin L, Stone K, et al (2023b) Llama 2: Open foundation and fine-tuned chat models. *CoRR* abs/2307.09288
- Vaswani A, Shazeer N, Parmar N, et al (2017) Attention is all you need. In: *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pp 5998–6008
- Vu T, Iyyer M, Wang X, et al (2023) Freshllms: Refreshing large language models with search engine augmentation. *CoRR* abs/2310.03214
- Wang B, Ping W, McAfee L, et al (2023a) Instructretro: Instruction tuning post retrieval-augmented pretraining. *CoRR* abs/2310.07713
- Wang B, Ping W, Xu P, et al (2023b) Shall we pretrain autoregressive language models with retrieval? A comprehensive study. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp 7763–7786
- Wang J, Yi X, Guo R, et al (2021a) Milvus: A purpose-built vector data management system. In: *SIGMOD '21: International Conference on Management of Data*. ACM, pp 2614–2627
- Wang L, Ma C, Feng X, et al (2024) A survey on large language model based autonomous agents. *Frontiers Comput Sci* 18(6):186345
- Wang S, Xu Y, Fang Y, et al (2022) Training data is more valuable than you think: A simple and effective method by retrieving from training data. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp 3170–3179
- Wang S, Zhu Y, Liu H, et al (2023c) Knowledge editing for large language models: A survey. *CoRR* abs/2310.16218
- Wang W, Dong L, Cheng H, et al (2023d) Augmenting language models with long-term memory. In: *Advances in Neural Information Processing Systems 36 (NeurIPS)*
- Wang X, Jiang Y, Bach N, et al (2021b) Improving named entity recognition by external context retrieving and cooperative learning. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th*

- International Joint Conference on Natural Language Processing (ACL/IJCNLP), pp 1800–1812
- Wang Y, Li P, Sun M, et al (2023e) Self-knowledge guided retrieval augmentation for large language models. In: Findings of the Association for Computational Linguistics (EMNLP), pp 10303–10315
- Wang Z, Araki J, Jiang Z, et al (2023f) Learning to filter context for retrieval-augmented generation. CoRR abs/2311.08377
- Wang Z, Nie W, Qiao Z, et al (2023g) Retrieval-based controllable molecule generation. In: The Eleventh International Conference on Learning Representations, (ICLR)
- Wu M, Goodman ND (2018) Multimodal generative models for scalable weakly-supervised learning. In: Advances in Neural Information Processing Systems 31 (NeurIPS), pp 5580–5590
- Wu S, Xiong Y, Cui Y, et al (2024) ReFusion: improving natural language understanding with computation-efficient retrieval representation fusion. CoRR abs/2401.02993
- Wu Y, Rabe MN, Hutchins D, et al (2022) Memorizing transformers. In: The Tenth International Conference on Learning Representations (ICLR)
- Xi Z, Chen W, Guo X, et al (2023) The rise and potential of large language model based agents: A survey. CoRR abs/2309.07864
- Xiao S, Liu Z, Zhang P, et al (2023) C-pack: Packaged resources to advance general chinese embedding. CoRR abs/2309.07597
- Xie J, Chen Z, Zhang R, et al (2024) Large multimodal agents: A survey. CoRR abs/2402.15116
- Xiong G, Jin Q, Lu Z, et al (2024) Benchmarking retrieval-augmented generation for medicine. In: Ku LW, Martins A, Srikumar V (eds) Findings of the Association for Computational Linguistics: ACL 2024
- Xiong Y, Yang X, Liu L, et al (2023) EARA: improving biomedical semantic textual similarity with entity-aligned attention and retrieval augmentation. In: Findings of the Association for Computational Linguistics (EMNLP). Association for Computational Linguistics, pp 8760–8771
- Xu F, Shi W, Choi E (2023a) RECOMP: improving retrieval-augmented lms with compression and selective augmentation. CoRR abs/2310.04408
- Xu FF, Alon U, Neubig G (2023b) Why do nearest neighbor language models work? In: Proceedings of the 40th International Conference on Machine Learning (ICML), pp 38325–38341

- Yan S, Gu J, Zhu Y, et al (2024) Corrective retrieval augmented generation. CoRR abs/2401.15884
- Yu G, Liu L, Jiang H, et al (2023a) Retrieval-augmented few-shot text classification. In: Findings of the Association for Computational Linguistics (EMNLP), pp 6721–6735
- Yu Z, Xiong C, Yu S, et al (2023b) Augmentation-adapted retriever improves generalization of language models as generic plug-in. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 2421–2436
- Zeng A, Liu X, Du Z, et al (2023) GLM-130B: an open bilingual pre-trained model. In: The Eleventh International Conference on Learning Representations (ICLR). OpenReview.net
- Zhang B, Sennrich R (2019) Root mean square layer normalization. In: Advances in Neural Information Processing Systems 32 (NeurIPS), pp 12360–12371
- Zhang H, Chen J, Jiang F, et al (2023a) Huatuoogpt, towards taming language model to be a doctor. In: Findings of the Association for Computational Linguistics (EMNLP). Association for Computational Linguistics, pp 10859–10885
- Zhang J, Muhamed A, Anantharaman A, et al (2023b) Reaugkd: Retrieval-augmented knowledge distillation for pre-trained language models. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 1128–1136
- Zhang N, Yao Y, Tian B, et al (2024a) A comprehensive study of knowledge editing for large language models. CoRR abs/2401.01286
- Zhang Y, Mao S, Ge T, et al (2024b) LLM as a mastermind: A survey of strategic reasoning with large language models. CoRR abs/2404.01230
- Zhang Z, Bo X, Ma C, et al (2024c) A survey on the memory mechanism of large language model based agents. CoRR abs/2404.13501
- Zhao P, Zhang H, Yu Q, et al (2024) Retrieval-augmented generation for ai-generated content: A survey. CoRR abs/2402.19473
- Zheng X, Zhang Z, Guo J, et al (2021) Adaptive nearest neighbor machine translation. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL/IJCNLP), pp 368–374
- Zhong Z, Lei T, Chen D (2022) Training language models with memory augmentation. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 5657–5673

Zhu W, Xu J, Huang S, et al (2023) INK: injecting knn knowledge in nearest neighbor machine translation. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp 15948–15959